

Collaborators/funders:

Systems and Software Security / FM Research Group

ARM Centre of Excellence

PPGEE, PPGI – UFAM

Centre for Digital Trust and Society

UKRI, EPSRC, EU Horizon and industrial partners

An Exploration of Automated Software Testing, Verification, and Repair using ESBMC and ChatGPT

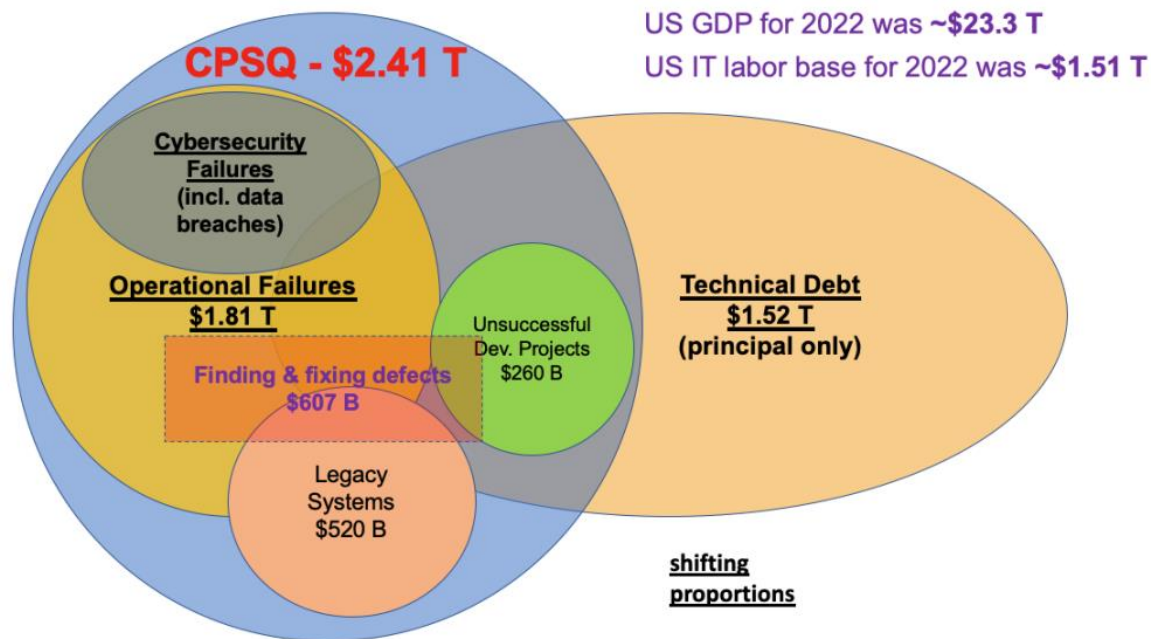


Lucas Cordeiro

lucas.cordeiro@manchester.ac.uk
<https://ssvlab.github.io/lucascordeiro/>

How much could software errors cost your business?

Poor software quality cost US companies **\$2.41 trillion** in 2022, while the **accumulated software Technical Debt (TD)** has grown to **~\$1.52 trillion**



TD relies on temporary easy-to-implement solutions to achieve short-term results at the expense of efficiency in the long run

The cost of poor software quality in the US: A 2022 Report

Objective of this talk

Discuss **automated testing, verification, and repair** techniques to establish a **robust foundation for building secure software systems**

- Introduce a **logic-based automated reasoning platform** to find and repair **software vulnerabilities**
- Explain **testing, verification, and repair** techniques to build **secure software systems**
- Present recent advancements towards a **hybrid approach** to protecting against **memory safety and concurrency vulnerabilities**

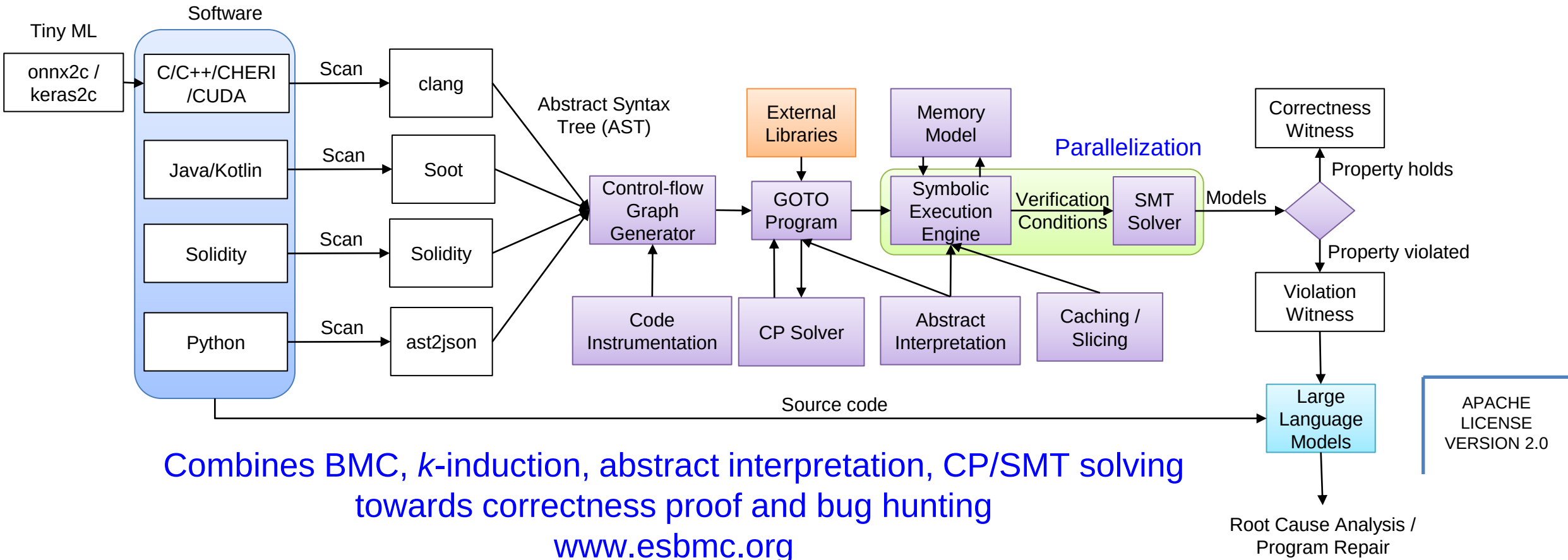
Research Questions

Given a **program** and a **safety/security specification**, can we automatically **verify** that the **program performs as specified**?

Can we leverage **program analysis/synthesis** to **discover and fix** more **software vulnerabilities** than existing state-of-the-art approaches?

ESBMC: An Automated Reasoning Platform

Logic-based automated reasoning for checking the **safety** and **security** of **AI** and software systems



The Bitter Lesson by Rich Sutton

March 13, 2019

“The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin. The ultimate reason for this is Moore's law, or rather its generalization of continued exponentially falling cost per unit of computation”

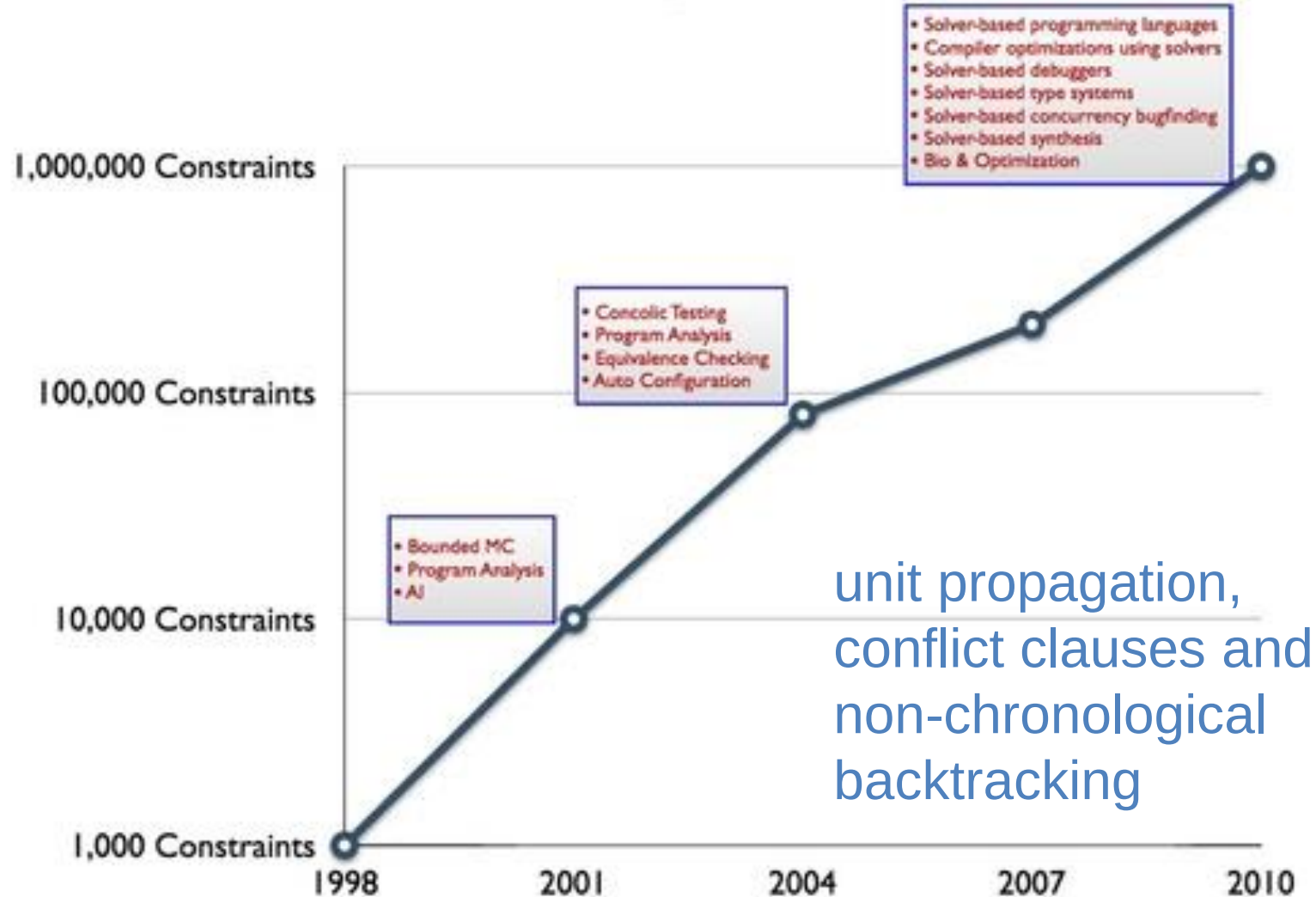
“The two methods that seem to scale arbitrarily in this way are *search* and *learning*”

Agenda

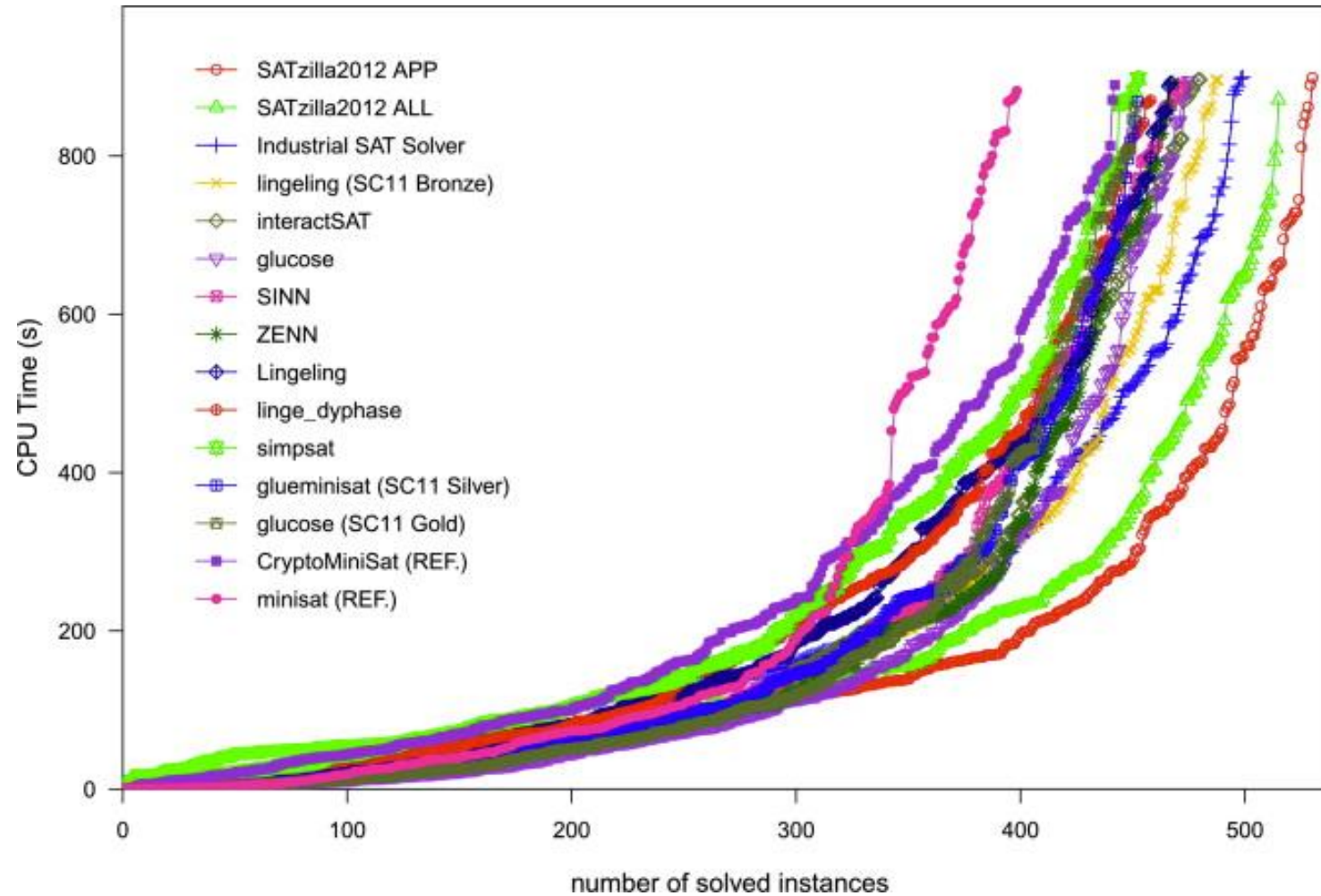
- Software Verification and Testing with the ESBMC Framework
- Towards Self-Healing Software via Large Language Models and Formal Verification
- Towards Verification of Programs for CHERI Platforms with ESBMC

SAT solving as enabling technology

SAT/SMT Solver Research Story A 1000x Improvement

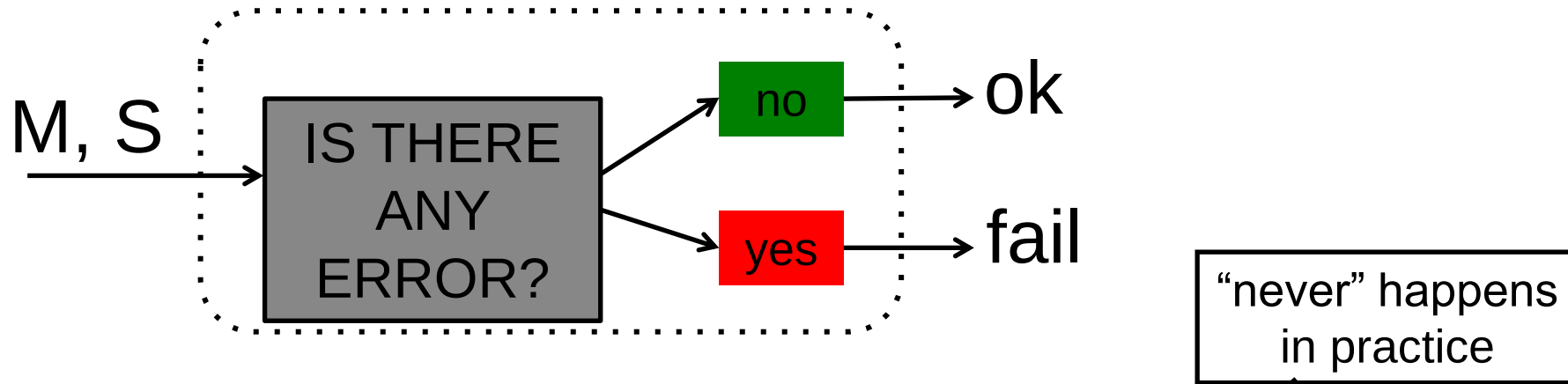


SAT Competition

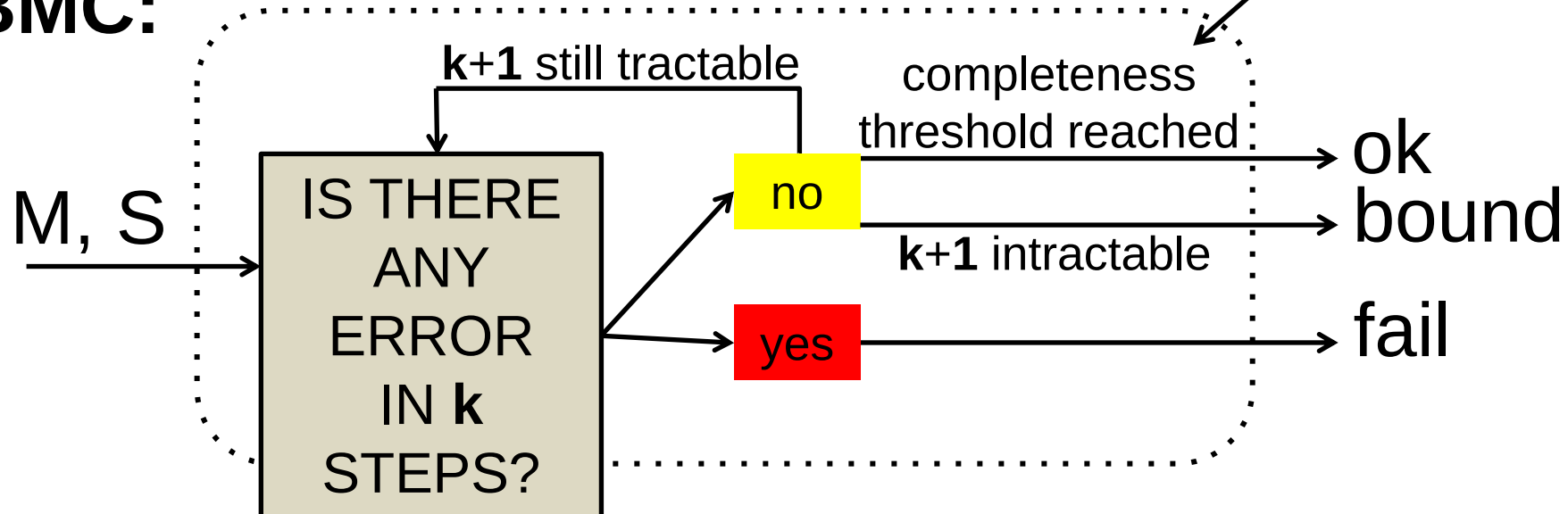


Bounded Model Checking (BMC)

MC:



BMC:

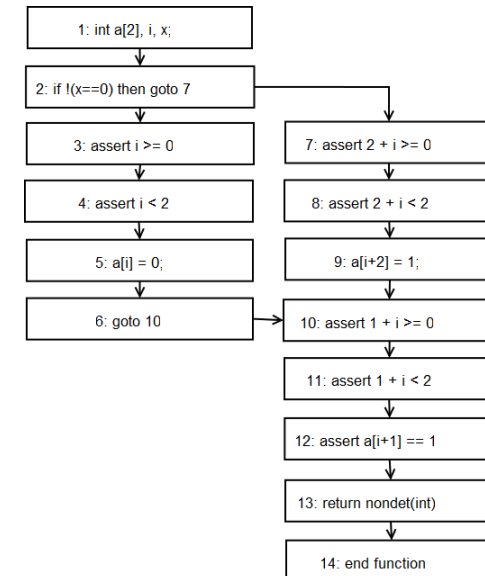


Software BMC

- program modelled as transition system
 - *state*: *pc* and program variables
 - derived from control-flow graph

```
int getPassword() {  
    char buf[2];  
    gets(buf);  
    return strcmp(buf, "ML");  
}
```

```
void main(){  
    int x=getPassword();  
    if(x){  
        printf("Access Denied\n");  
        exit(0);  
    }  
    printf("Access Granted\n");  
}
```

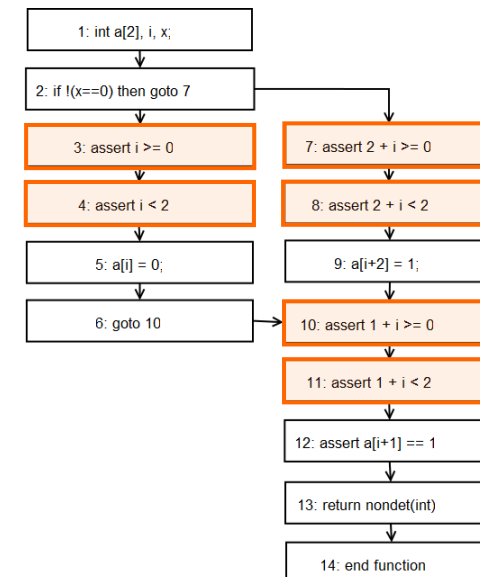
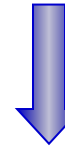


Software BMC

- program modelled as transition system
 - *state*: *pc* and program variables
 - derived from control-flow graph
 - added safety properties as extra nodes

```
int getPassword() {  
    char buf[2];  
    gets(buf);  
    return strcmp(buf, "ML");  
}
```

```
void main(){  
    int x=getPassword();  
    if(x){  
        printf("Access Denied\n");  
        exit(0);  
    }  
    printf("Access Granted\n");  
}
```

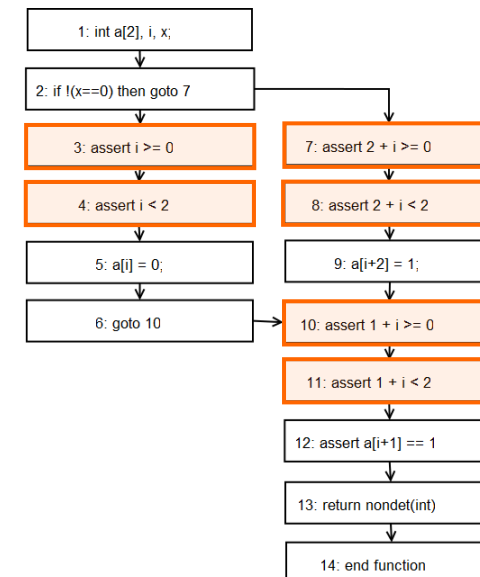
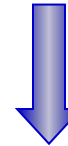


Software BMC

- program modelled as transition system
 - *state*: *pc* and program variables
 - derived from control-flow graph
 - added safety properties as extra nodes
- program unfolded up to given bounds

```
int getPassword() {  
    char buf[2];  
    gets(buf);  
    return strcmp(buf, "ML");  
}
```

```
void main(){  
    int x=getPassword();  
    if(x){  
        printf("Access Denied\n");  
        exit(0);  
    }  
    printf("Access Granted\n");  
}
```



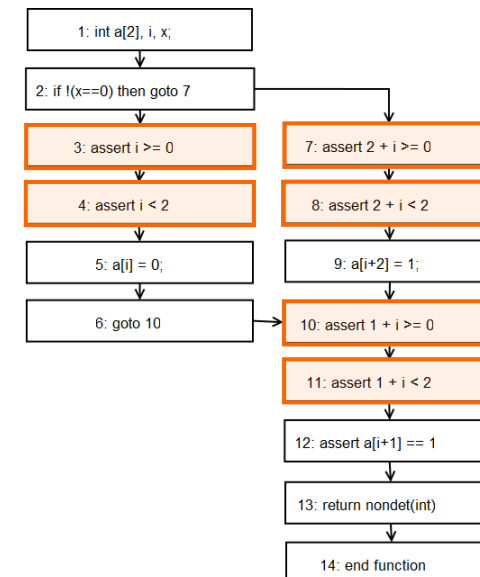
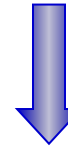
Software BMC

- program modelled as transition system
 - *state*: *pc* and program variables
 - derived from control-flow graph
 - added safety properties as extra nodes
- program unfolded up to given bounds
- unfolded program optimized to reduce blow-up
 - constant propagation/slicing
 - forward substitutions/caching
 - unreachable code/pointer analysis

} crucial

```
int getPassword() {  
    char buf[2];  
    gets(buf);  
    return strcmp(buf, "ML");  
}
```

```
void main(){  
    int x=getPassword();  
    if(x){  
        printf("Access Denied\n");  
        exit(0);  
    }  
    printf("Access Granted\n");  
}
```



Software BMC

- program modelled as transition system
 - *state*: *pc* and program variables
 - derived from control-flow graph
 - added safety properties as extra nodes
- program unfolded up to given bounds
- unfolded program optimized to reduce blow-up
 - constant propagation/slicing
 - forward substitutions/caching
 - unreachable code/pointer analysis

} crucial
- front-end converts unrolled and **optimized program into SSA**

```
int getPassword() {  
    char buf[2];  
    gets(buf);  
    return strcmp(buf, "ML");  
}
```

```
void main(){  
    int x=getPassword();  
    if(x){  
        printf("Access Denied\n");  
        exit(0);  
    }  
    printf("Access Granted\n");  
}
```



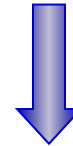
```
g1 = x1 == 0  
a1 = a0 WITH [i0:=0]  
a2 = a0  
a3 = a2 WITH [2+i0:=1]  
a4 = g1 ? a1 : a3  
t1 = a4 [1+i0] == 1
```

Software BMC

- program modelled as transition system
 - *state*: *pc* and program variables
 - derived from control-flow graph
 - added safety properties as extra nodes
- program unfolded up to given bounds
- unfolded program optimized to reduce blow-up
 - constant propagation/slicing
 - forward substitutions/caching
 - unreachable code/pointer analysis
- front-end converts unrolled and **optimized program into SSA**
- extraction of *constraints C* and *properties P*

```
int getPassword() {  
    char buf[2];  
    gets(buf);  
    return strcmp(buf, "ML");  
}
```

```
void main(){  
    int x=getPassword();  
    if(x){  
        printf("Access Denied\n");  
        exit(0);  
    }  
    printf("Access Granted\n");  
}
```



$$C := \left[\begin{array}{l} g_1 := (x_1 = 0) \\ \wedge a_1 := \text{store}(a_0, i_0, 0) \\ \wedge a_2 := a_0 \\ \wedge a_3 := \text{store}(a_2, 2 + i_0, 1) \\ \wedge a_4 := \text{ite}(g_1, a_1, a_3) \end{array} \right]$$

$$P := \left[\begin{array}{l} i_0 \geq 0 \wedge i_0 < 2 \\ \wedge 2 + i_0 \geq 0 \wedge 2 + i_0 < 2 \\ \wedge 1 + i_0 \geq 0 \wedge 1 + i_0 < 2 \\ \wedge \text{select}(a_4, i_0 + 1) = 1 \end{array} \right]$$

Software BMC

- program modelled as transition system
 - *state*: *pc* and program variables
 - derived from control-flow graph
 - added safety properties as extra nodes
- program unfolded up to given bounds
- unfolded program optimized to reduce blow-up
 - constant propagation/slicing
 - forward substitutions/caching
 - unreachable code/pointer analysis
- front-end converts unrolled and **optimized program into SSA**
- extraction of *constraints C* and *properties P*
 - specific to selected SMT solver, uses theories

```
int getPassword() {  
    char buf[2];  
    gets(buf);  
    return strcmp(buf, "ML");  
}
```

```
void main(){  
    int x=getPassword();  
    if(x){  
        printf("Access Denied\n");  
        exit(0);  
    }  
    printf("Access Granted\n");  
}
```



$$C := \left[\begin{array}{l} g_1 := (x_1 = 0) \\ \wedge a_1 := \text{store}(a_0, i_0, 0) \\ \wedge a_2 := a_0 \\ \wedge a_3 := \text{store}(a_2, 2 + i_0, 1) \\ \wedge a_4 := \text{ite}(g_1, a_1, a_3) \end{array} \right]$$

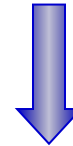
$$P := \left[\begin{array}{l} i_0 \geq 0 \wedge i_0 < 2 \\ \wedge 2 + i_0 \geq 0 \wedge 2 + i_0 < 2 \\ \wedge 1 + i_0 \geq 0 \wedge 1 + i_0 < 2 \\ \wedge \text{select}(a_4, i_0 + 1) = 1 \end{array} \right]$$

Software BMC

- program modelled as transition system
 - *state*: *pc* and program variables
 - derived from control-flow graph
 - added safety properties as extra nodes
- program unfolded up to given bounds
- unfolded program optimized to reduce blow-up
 - constant propagation/slicing
 - forward substitutions/caching
 - unreachable code/pointer analysis
- front-end converts unrolled and **optimized program into SSA**
- extraction of *constraints C* and *properties P*
 - specific to selected SMT solver, uses theories
- satisfiability check of $C \wedge \neg P$

```
int getPassword() {  
    char buf[2];  
    gets(buf);  
    return strcmp(buf, "ML");  
}
```

```
void main(){  
    int x=getPassword();  
    if(x){  
        printf("Access Denied\n");  
        exit(0);  
    }  
    printf("Access Granted\n");  
}
```


$$C := \left[\begin{array}{l} g_1 := (x_1 = 0) \\ \wedge a_1 := store(a_0, i_0, 0) \\ \wedge a_2 := a_0 \\ \wedge a_3 := store(a_2, 2 + i_0, 1) \\ \wedge a_4 := ite(g_1, a_1, a_3) \end{array} \right]$$
$$P := \left[\begin{array}{l} i_0 \geq 0 \wedge i_0 < 2 \\ \wedge 2 + i_0 \geq 0 \wedge 2 + i_0 < 2 \\ \wedge 1 + i_0 \geq 0 \wedge 1 + i_0 < 2 \\ \wedge select(a_4, i_0 + 1) = 1 \end{array} \right]$$

Induction-Based Verification for Software

k -induction checks loop-free programs...

- **base case** ($base_k$): find a counter-example with up to k loop unwindings (plain BMC)
- **forward condition** (fwd_k): check that P holds in all states reachable within k unwindings
- **inductive step** ($step_k$): check that whenever P holds for k unwindings, it also holds after next unwinding
 - havoc variables
 - assume loop condition
 - run loop body (k times)
 - assume loop termination

⇒ iterative deepening if inconclusive

Induction-Based Verification for Software

```
k=1
while k ≤ max_iterations do
  if  $base_{P,\phi,k}$  then
    return trace s[0..k]
  else
    k=k+1
    if  $fwd_{P,\phi,k}$  then
      return true
    else if  $step_{P',\phi,k}$  then
      return true
    end if
  end
return unknown
```

```
unsigned int x=*;
while(x>0) x--;
 $assume(x \leq 0);$ 
assert(x==0);
```

```
unsigned int x=*;
while(x>0) x--;
 $assert(x \leq 0);$ 
assert(x==0);
```

```
unsigned int x=*;
 $assume(x > 0);$ 
while(x>0) x--;
 $assume(x \leq 0);$ 
assert(x==0);
```

Automatic Invariant Generation

- Infer invariants based on **intervals** as abstract domain via a dependence graph
 - *E.g., $a \leq x \leq b$ (integer and floating-point)*
 - Inject intervals as assumptions and contract them via CSP
 - Remove unreachable states

Line	Interval for “a”	Restriction
4	$(-\infty, +\infty)$	None
6	$(-\infty, 100]$	$a \leq 100$
7	$(100, +\infty)$	$a > 100$

```
1 int main()
2 {
3     int a = *;
4
5     while(a <= 100)
6         a++;
7     assert(a>10);
8     return 0;
9 }
```

k-Induction proof rule
“hijacks” loop conditions
to nondeterministic
values, thus computing
intervals become
essential

***k*-Induction can prove the correctness of more programs when the invariant generation is enabled**

BMC of Software Using Interval Methods via Contractors

- 1) Analyze intervals and properties
 - Static Analysis / Abstract Interpretation
- 2) Convert the problem into a CSP
 - Variables, Domains and Constraints
- 3) Apply contractor to CSP
 - Forward-Backward Contractor
- 4) Apply reduced intervals back to the program

```

1 unsigned int x=nondet_uint();
2 unsigned int y=nondet_uint();
3 __ESBMC_assume(x >= 20 && x <= 30);
4 __ESBMC_assume(y <= 30);
5 assert(x >= y);
    
```

```
__ESBMC_assume(y <= 30 && y >= 20);
```

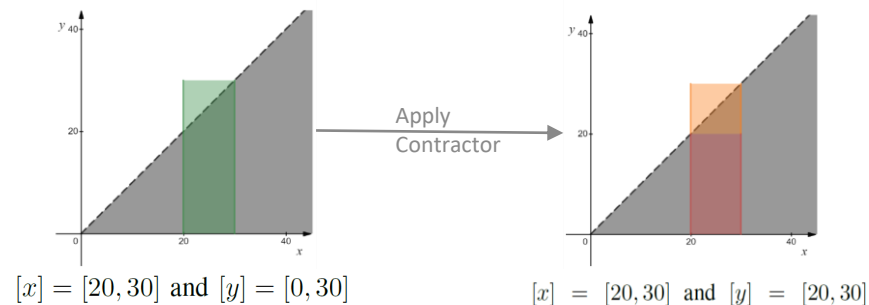
This **assumption** prunes our search space to the **orange area**

```

1 unsigned int x=nondet_uint();
2 unsigned int y=nondet_uint();
3 __ESBMC_assume(x >= 20 && x <= 30);
4 __ESBMC_assume(y <= 30);
5 assert(x >= y);
    
```

Domain: $[x] = [20, 30]$ and $[y] = [0, 30]$

Constraint: $y - x \leq 0$



$$f(x) > 0 \quad I = [0, \infty)$$

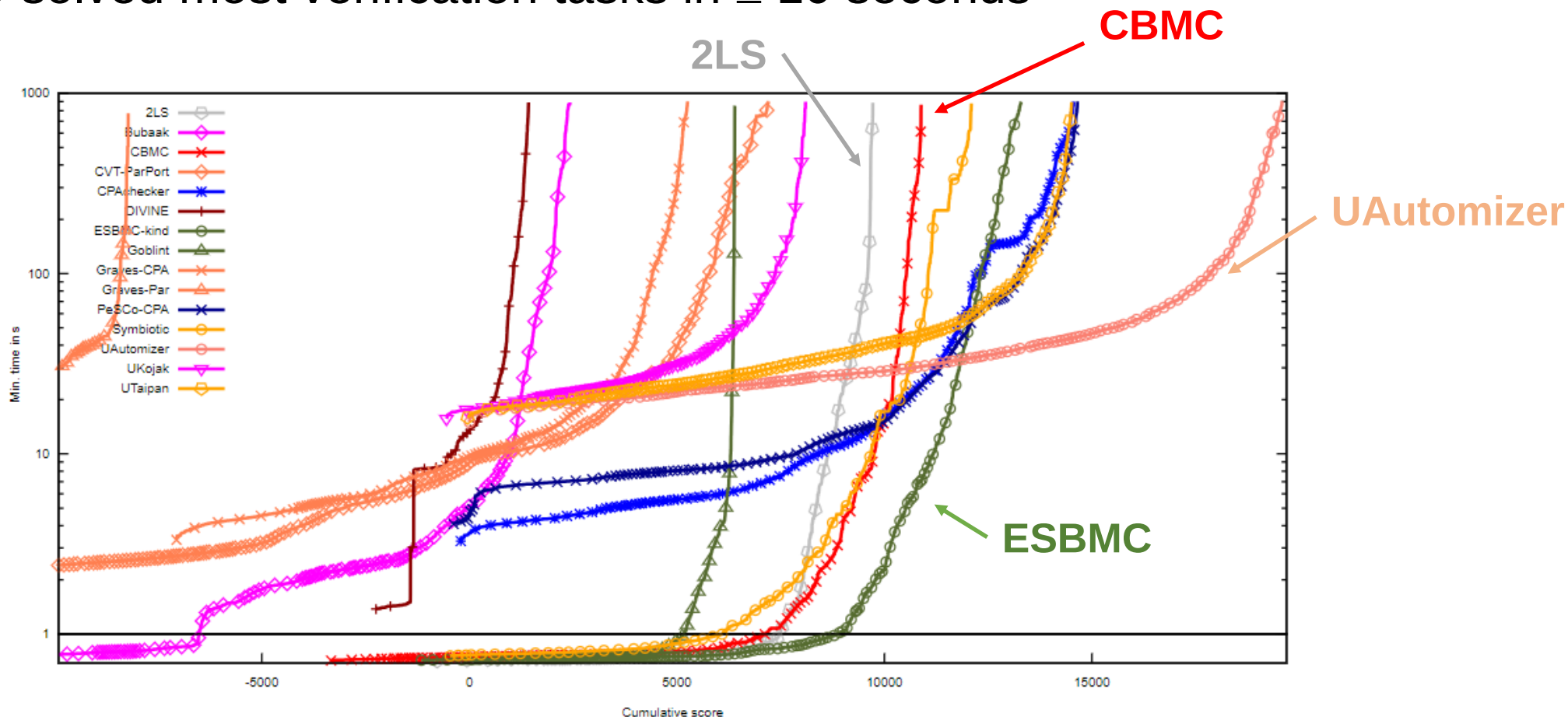
$$f(x) = y - x \quad [f(x)_1] = I \cap [y_0] - [x_0] \quad \text{Forward-step}$$

$$x = y - f(x) \quad [x_1] = [x_0] \cap [y_0] - [f(x)_1] \quad \text{Backward-step}$$

$$y = f(x) + x \quad [y_1] = [y_0] \cap [f(x)_1] + [x_1] \quad \text{Backward-step}$$

Intl. Software Verification Competition (SV-Comp 2023)

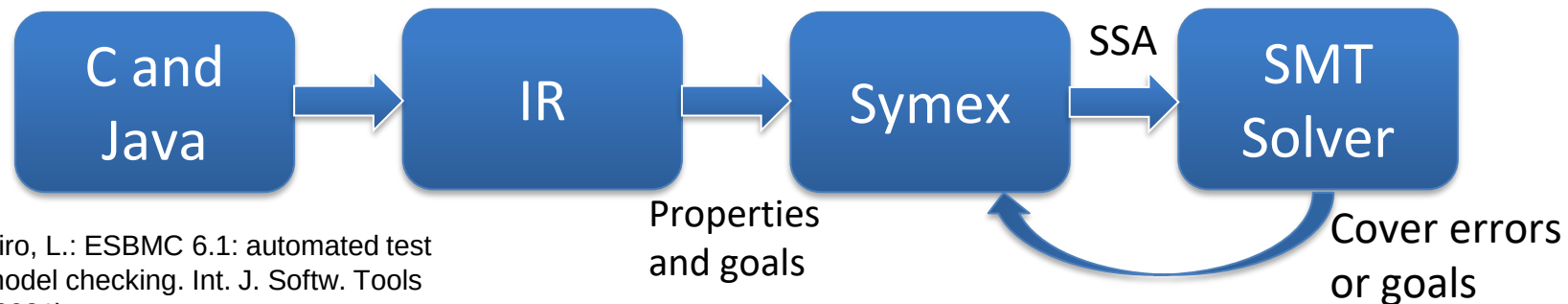
- SV-COMP 2023, 23805 verification tasks, max. score: 38644
- ESBMC solved most verification tasks in ≤ 10 seconds



Verification of the Overall Category

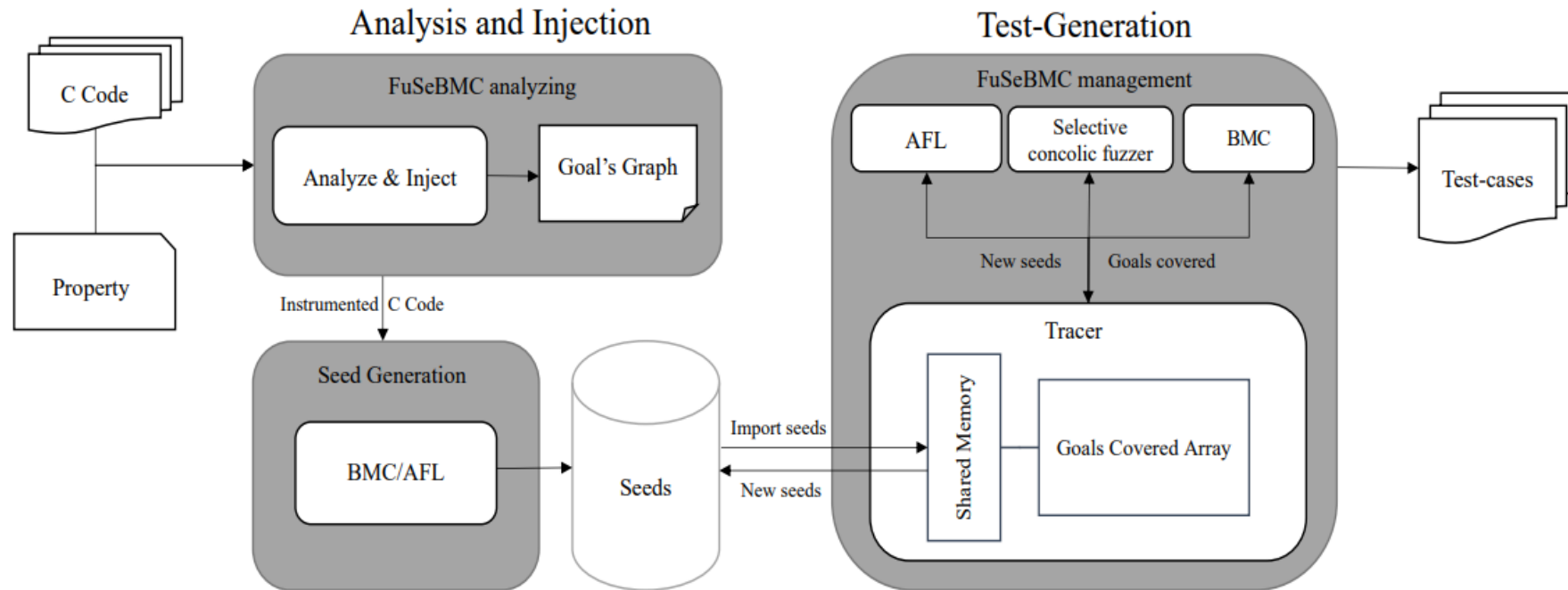
White-box Fuzzing: Bug Finding and Code Coverage

- Translate the program to an **intermediate representation** (IR)
- Add properties to check **errors** or goals to check **coverage**
- **Symbolically** execute IR to produce an SSA program
- Translate the resulting SSA program into a **logical formula**
- Solve the formula iteratively to cover errors and goals
- Interpret the solution to figure out the **input conditions**
- Spit those input conditions out as a test case



FuSeBMC v4 Framework

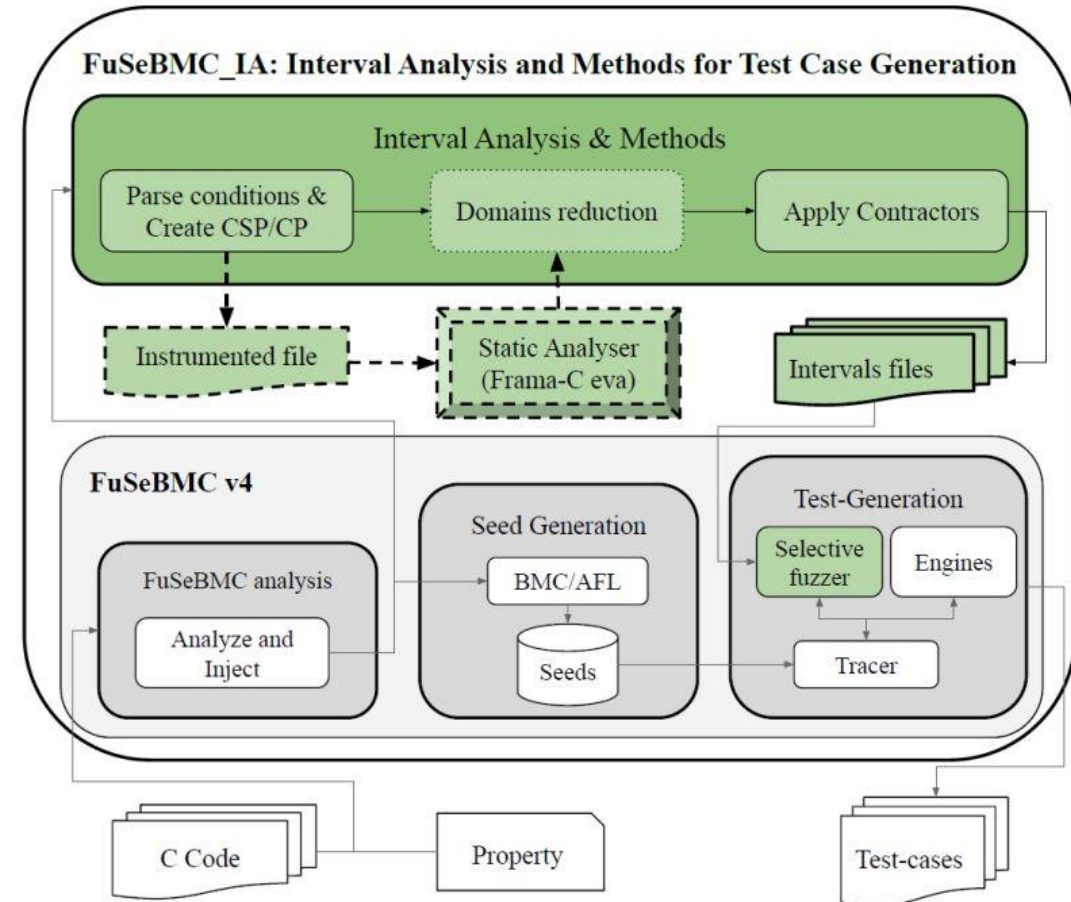
- Use **Clang** tooling infrastructure
- Employ three engines in its **reachability analysis: one BMC and two fuzzing engines**
- Use a **tracer** to coordinate the various engines



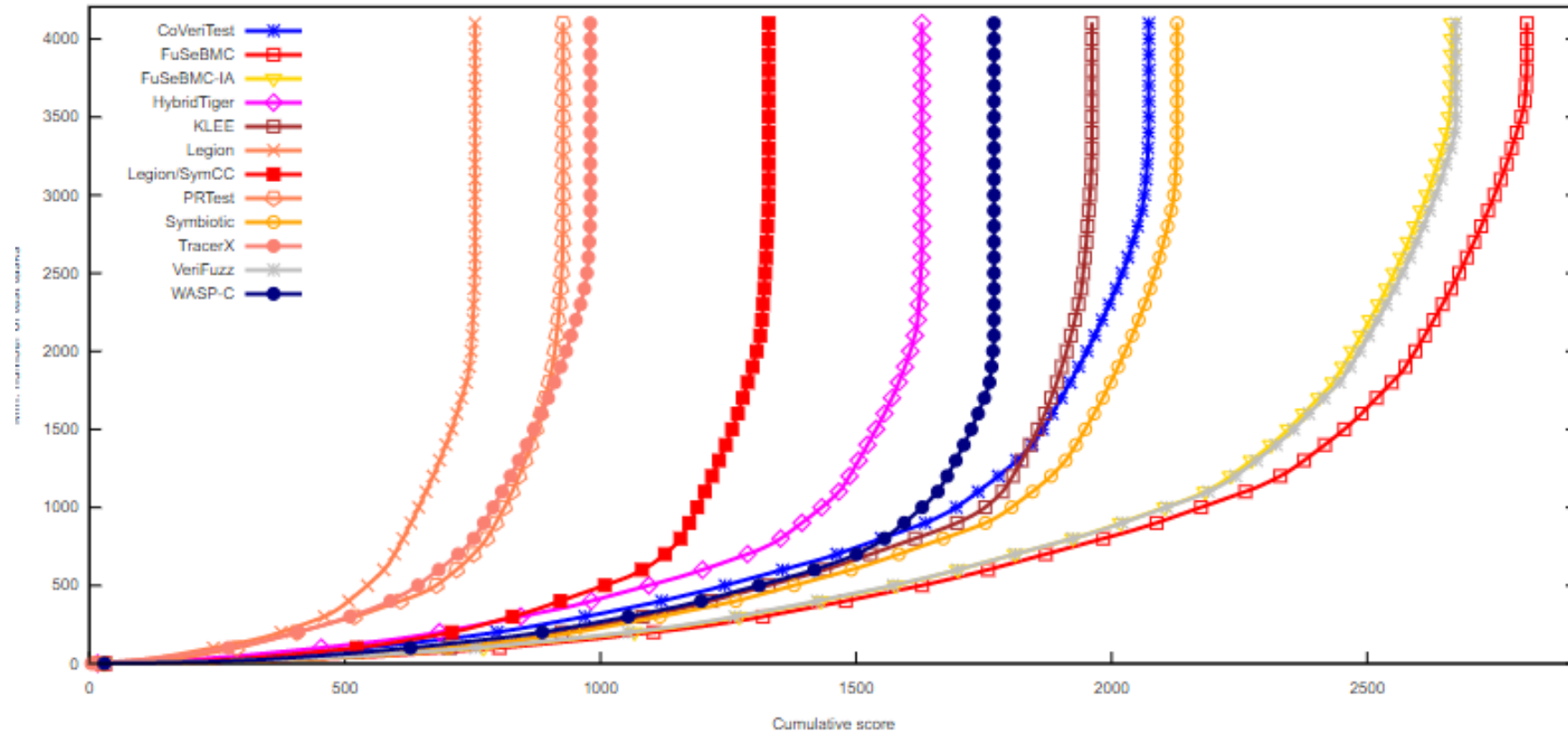
Interval Analysis and Methods for Automated Test Case Generation

This combined method can **reduce CPU time, memory usage, and energy consumption**

We advocate that combining **cooperative verification** and **constraint programming** is essential to leverage a **modular cooperative cloud-native testing platform**

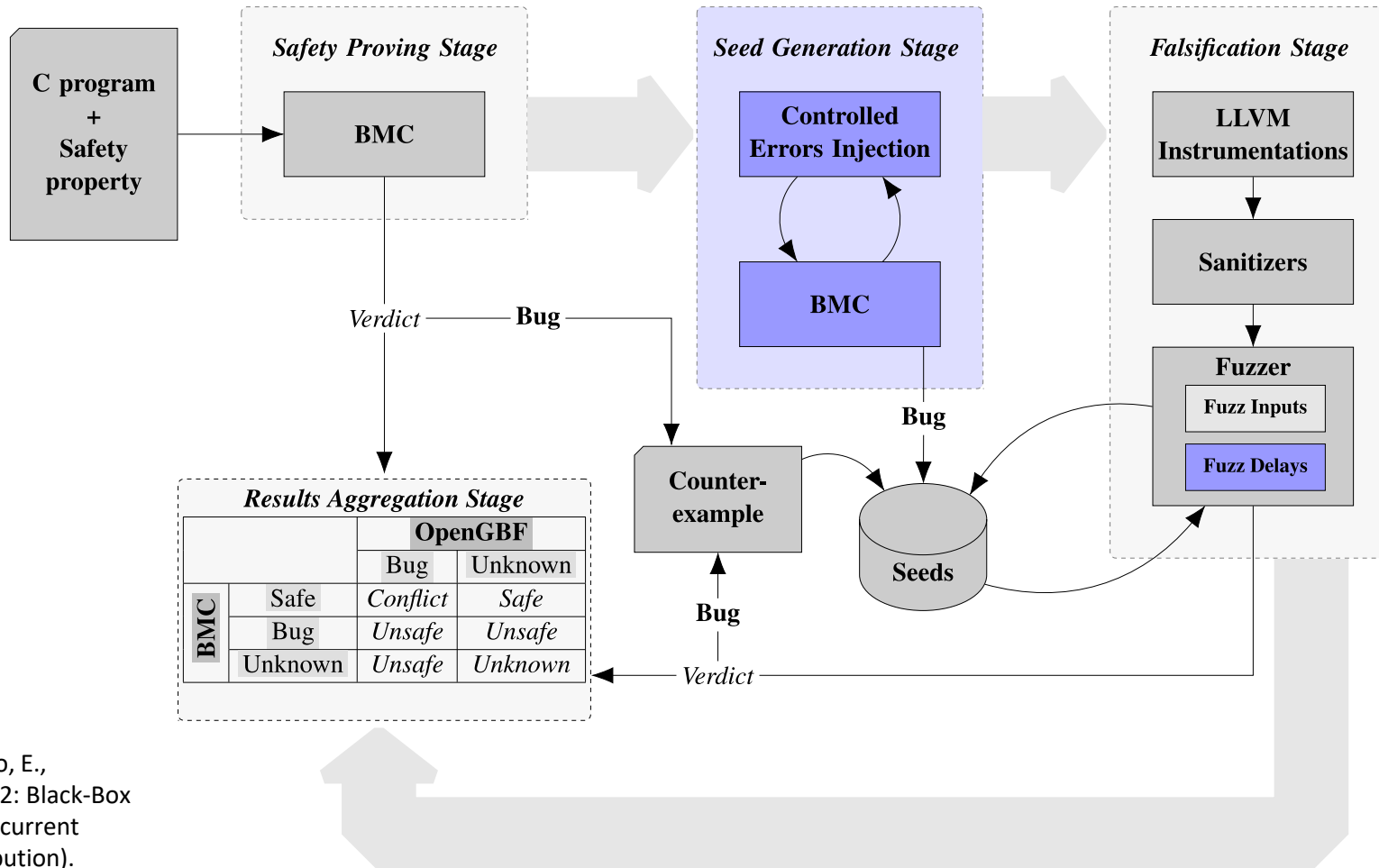


Competition on Software Testing 2023: Results of the Overall Category



FuSeBMC achieved 3 awards: 1st place in Cover-Error, 1st place in Cover-BrancheS, and 1st place in Overall

EBF: Black-Box Cooperative Verification for Concurrent Programs



EBF 4.0 with different BMC tools

- **BMC** 6 min + **OpenGBF** 5 min + **results Aggregation** 4 min = 15 min
- **RAM limit** is 15 GB per Benchexec run
- **ConcurrencySafety main** from SV-COMP 2022
 - Witness validation switched off
- Ubuntu 20.04.4 LTS with 160 GB RAM and 25 cores

Verification outcome	Tool							
	<i>EBF</i>	<i>Deagle</i>	<i>EBF</i>	<i>Cseq</i>	<i>EBF</i>	<i>ESBMC</i>	<i>EBF</i>	<i>CBMC</i>
Correct True	240	240	172	177	65	70	139	146
Correct False	336	319	333	313	308	268	320	303
Incorrect True	0	0	0	0	0	0	0	0
Incorrect False	0	0	0	0	0	1	0	3
Unknown	187	204	258	273	390	424	304	311

- EBF4.0 **increases** the number of **detected bugs** for BMC tools
- EBF4.0 provides a better **trade-off** between **bug finding** and **safety proving** than each BMC engine

WolfMQTT Verification

- **wolfMQTT** library is a client implementation of the MQTT protocol written in C for **IoT devices**

subscribe_task
and waitMessage_task are
called through different threads
accessing packet_ret,
causing a data race in
MqttClient_WaitType

Here is where the
data race might
happen! Unprotected
pointer

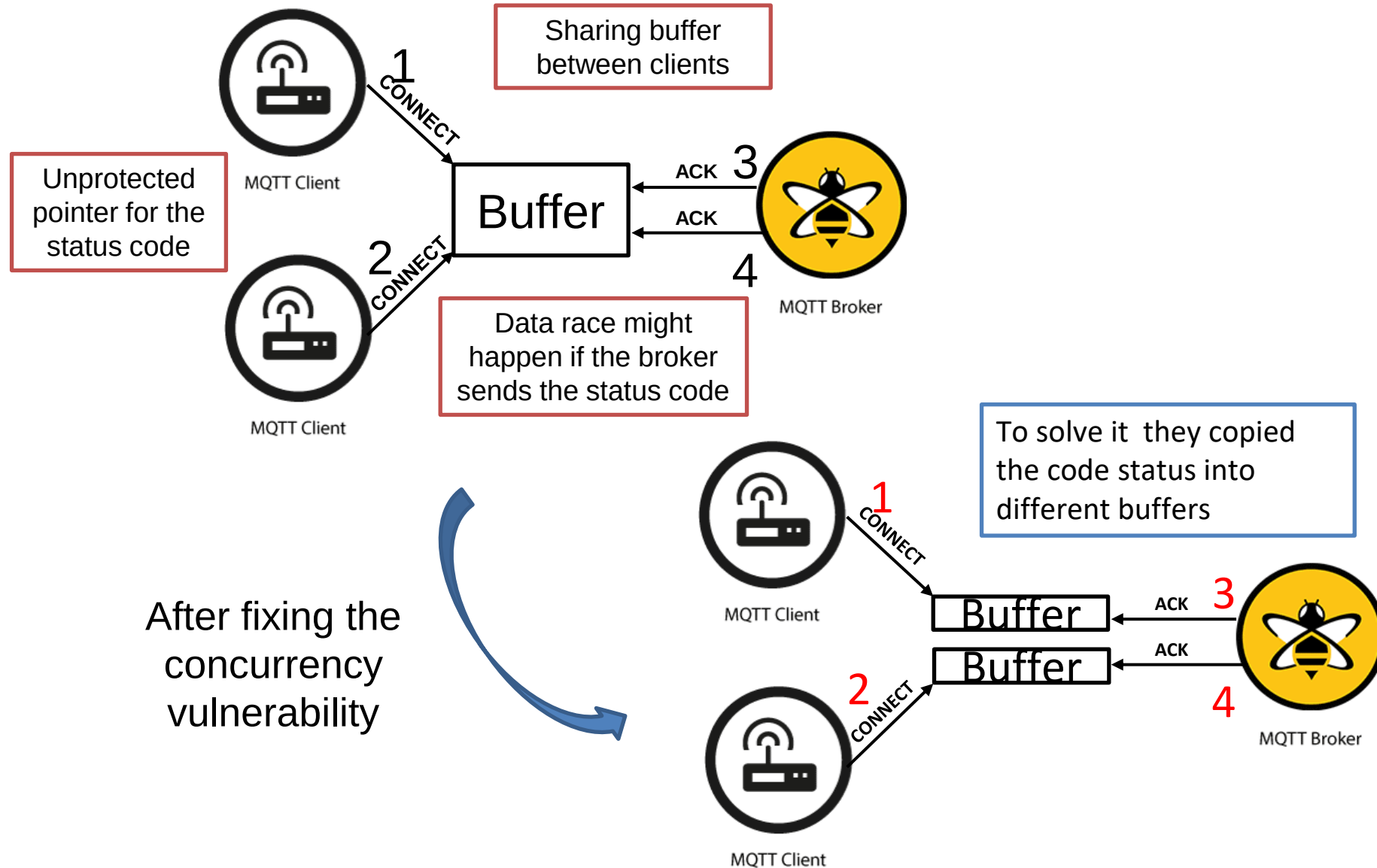
```
Int main(){
Pthread_t th1, th2;
static MQTTCtx mqttCtx;
pthread_create(&th1, subscribe_task, &mqttCtx))
pthread_create(&th2, waitMessage_task, &mqttCtx))}

static void *subscribe_task(void *client){
.....
MqttClient_WaitType(client,msg,MQTT_PACKET_TYPE_ANY,
0,timeout_ms);
.....}

static void *waitMessage_task(void *client){
...
MqttClient_WaitType(client, msg, MQTT_PACKET_TYPE_ANY,
0,timeout_ms);
.....}

static int MqttClient_WaitType(MqttClient *client,
void *packet_obj,
byte wait_type, word16 wait_packet_id, int timeout_ms)
{
.....
rc = wm_SemLock(&client->lockClient);
if (rc == 0) {
if (MqttClient_RespList_Find(client,
(MqttPacketType)wait_type,
wait_packet_id, &pendResp)) {
if (pendResp->packetDone) {
rc = pendResp->packet_ret;
.....}
.....}
```

WolfMQTT Verification



Bug Report

Fixes for multi-threading issues #209

<> Code

Merged embhorn merged 1 commit into wolfSSL:master from dgarske:mt_suback on 3 Jun 2021

Conversation 2 Commits 1 Checks 0 Files changed 4

+74 -48



dgarske commented on 2 Jun 2021

Contributor

1. The client lock is needed earlier to protect the "reset the packet state".
 2. The subscribe ack was using an unprotected pointer to response code list. Now it makes a copy of those codes.
 3. Add protection to multi-thread example "stop" variable.
- Thanks to Fatimah Aljaafari (@fatimahkj) for the report.
ZD 12379 and PR [Data race at function MqttClient_WaitType](#) #198

Fixes for three multi-thread issues: ...

78370ed

dgarske requested a review from embhorn 15 months ago

dgarske assigned embhorn on 2 Jun 2021



embhorn approved these changes on 3 Jun 2021

View changes

Reviewers

lygstate

embhorn

Assignees

embhorn

Labels

None yet

Projects

None yet

Milestone

No milestone

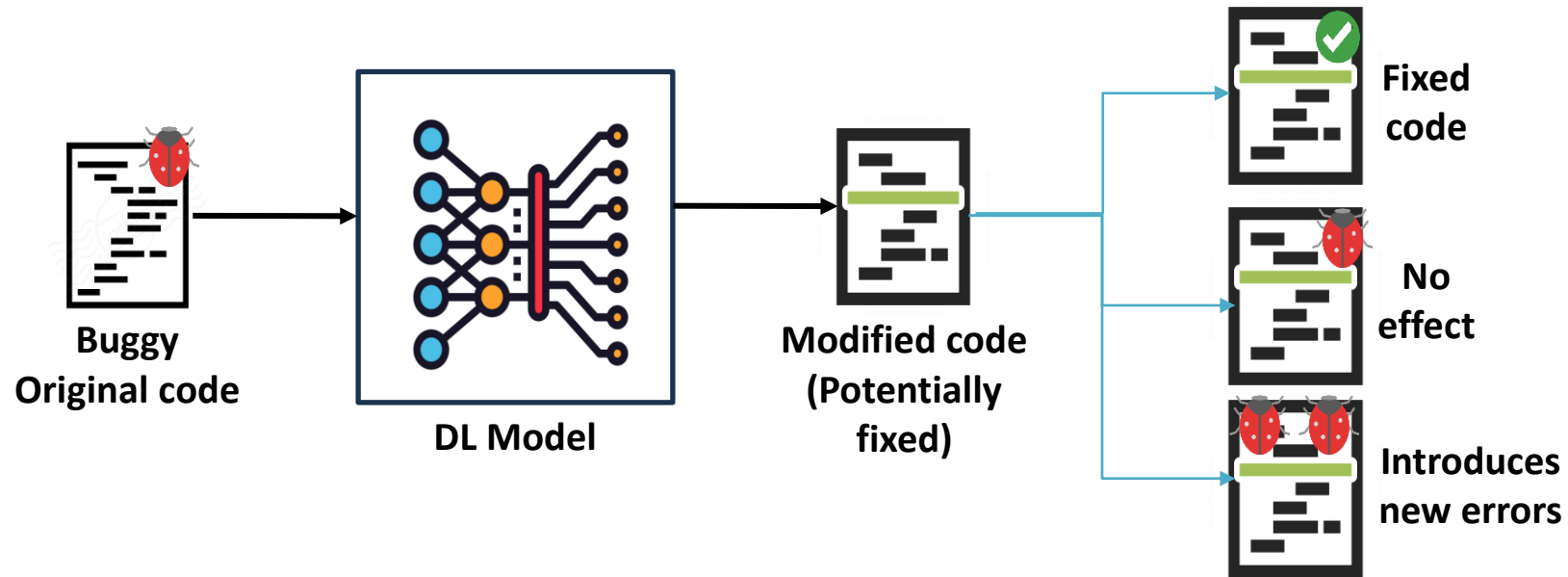
<https://github.com/wolfSSL/wolfMQTT>



Agenda

- Software Verification and Testing with the ESBMC Framework
- Towards Self-Healing Software via Large Language Models and Formal Verification
- Towards Verification of Programs for CHERI Platforms with ESBMC

Deep Learning and Automated Program Repair

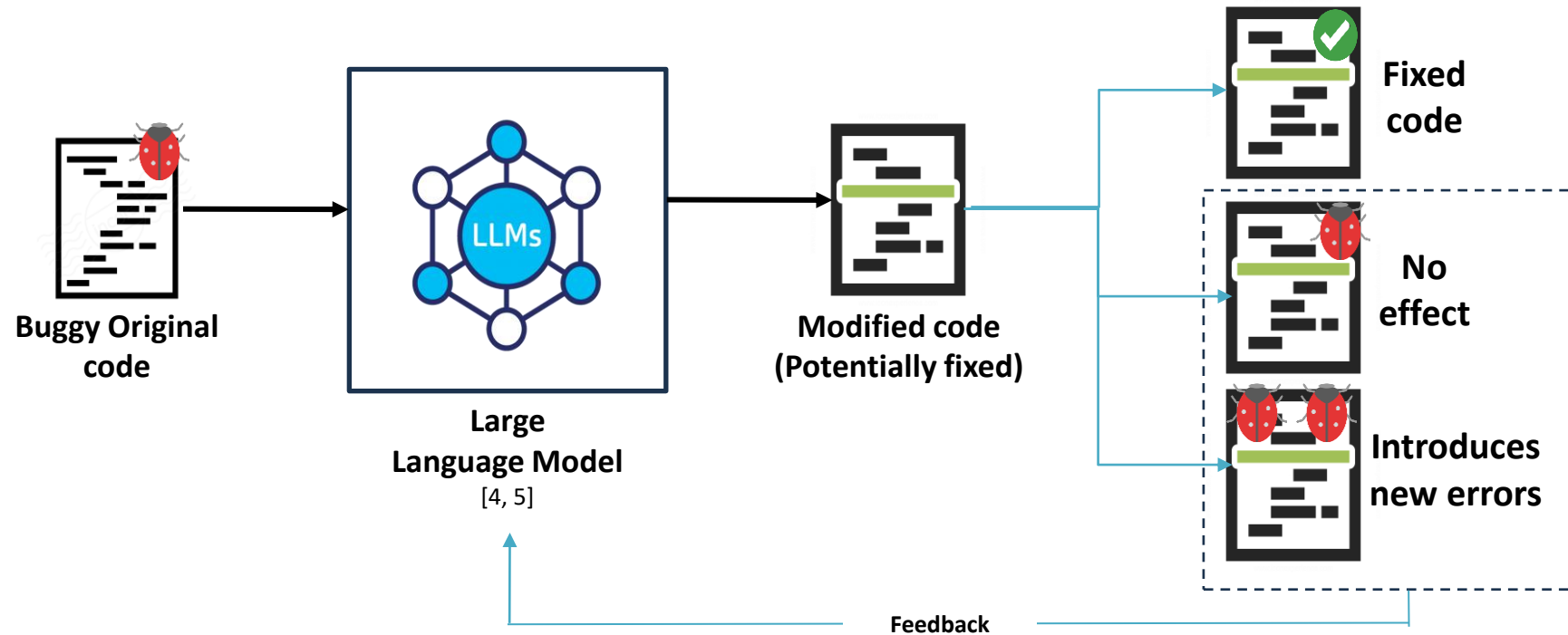


[1] Jin M, Shahriar S, Tufano M, Shi X, Lu S, Sundaresan N, Svyatkovskiy A. InferFix: End-to-End Program Repair with LLMs. arXiv e-prints. 2023 Mar:arXiv-2303.

[2] Li Y, Wang S, Nguyen TN. Dlfix: Context-based code transformation learning for automated program repair. In Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering 2020 Jun 27 (pp. 602-614).

[3] Gupta R, Pal S, Kanade A, Shevade S. Deepfix: Fixing common c language errors by deep learning. In Proceedings of the aaai conference on artificial intelligence 2017 Feb 12 (Vol. 31, No. 1).

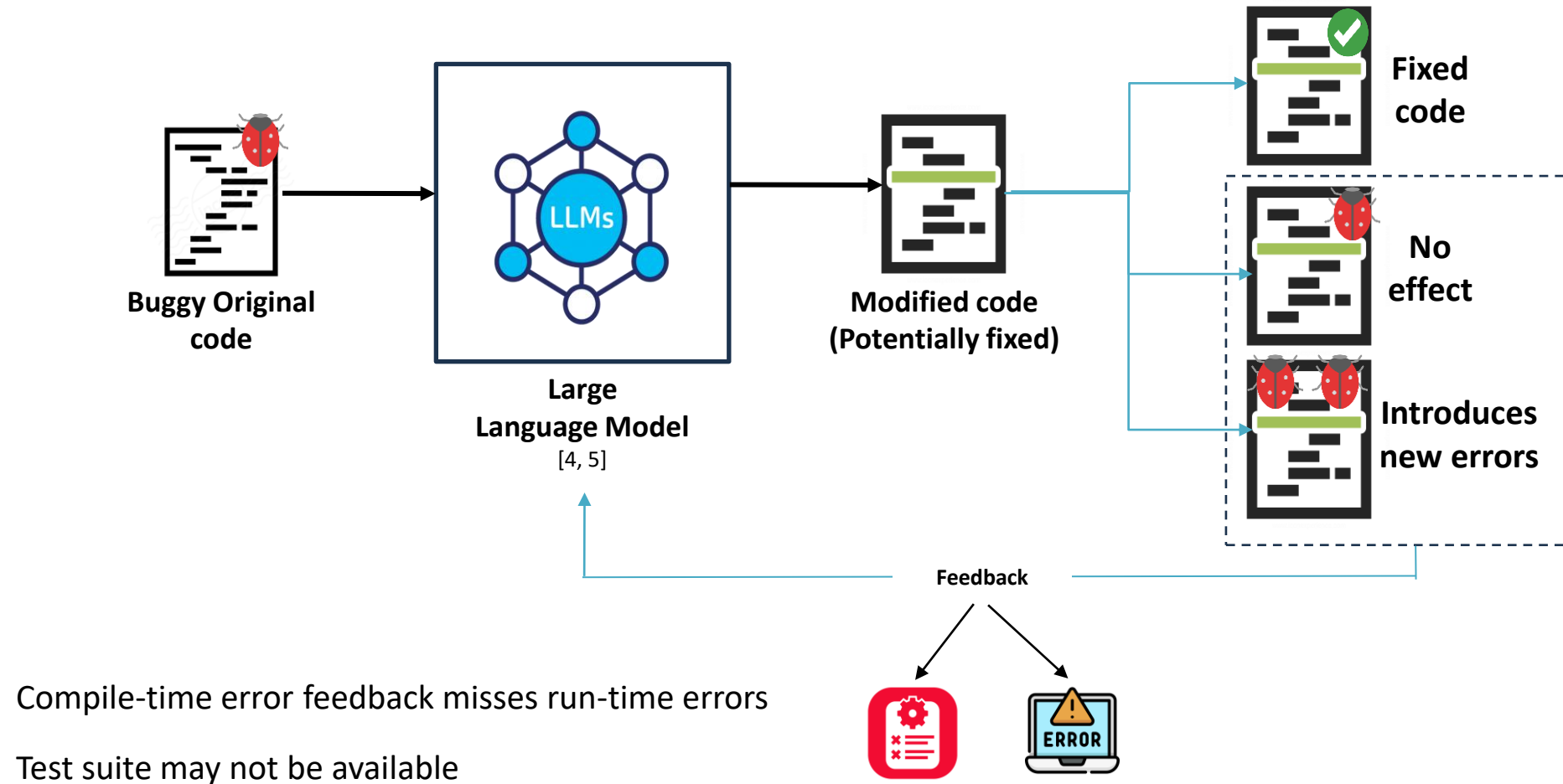
Large Language Models and Automated Program Repair



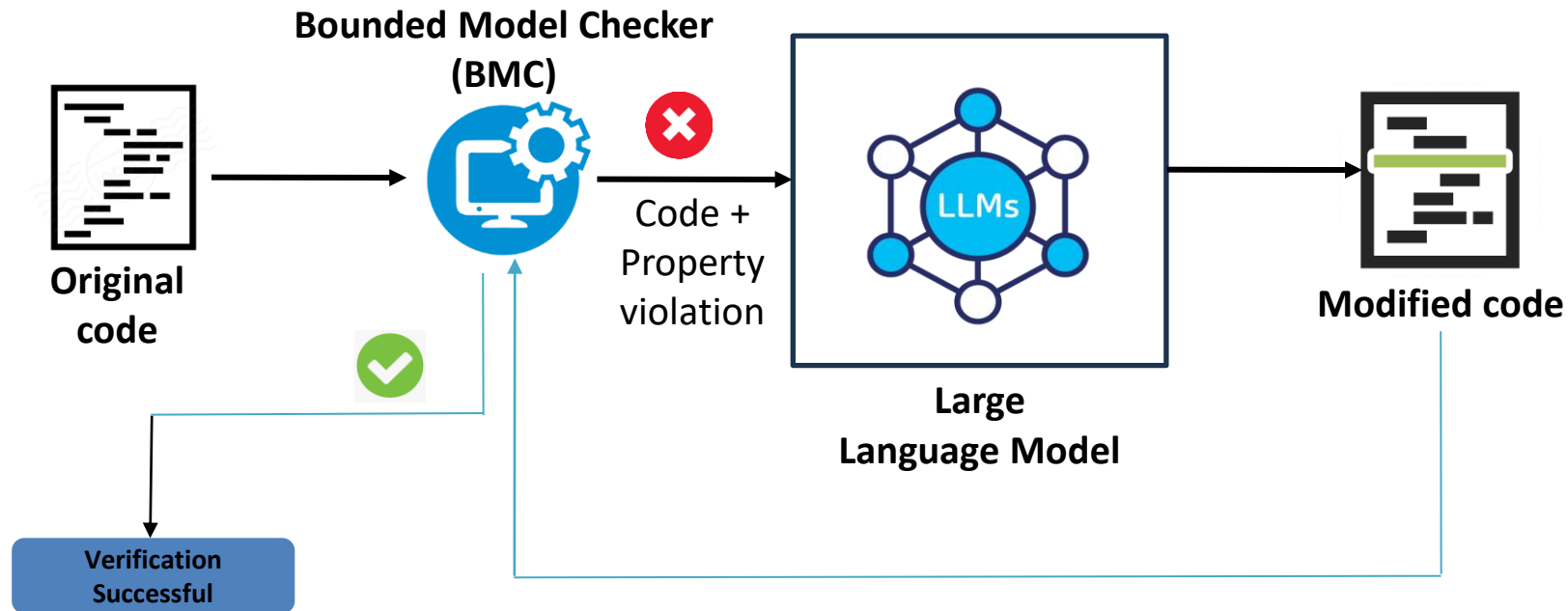
[4] Wang X, Wang Y, Wan Y, Mi F, Li Y, Zhou P, Liu J, Wu H, Jiang X, Liu Q. Compilable neural code generation with compiler feedback. arXiv preprint arXiv:2203.05132. 2022 Mar 10.

[5] Xia CS, Zhang L. Conversational automated program repair. arXiv preprint arXiv:2301.13246. 2023 Jan 30.

Large Language Models and Automated Program Repair

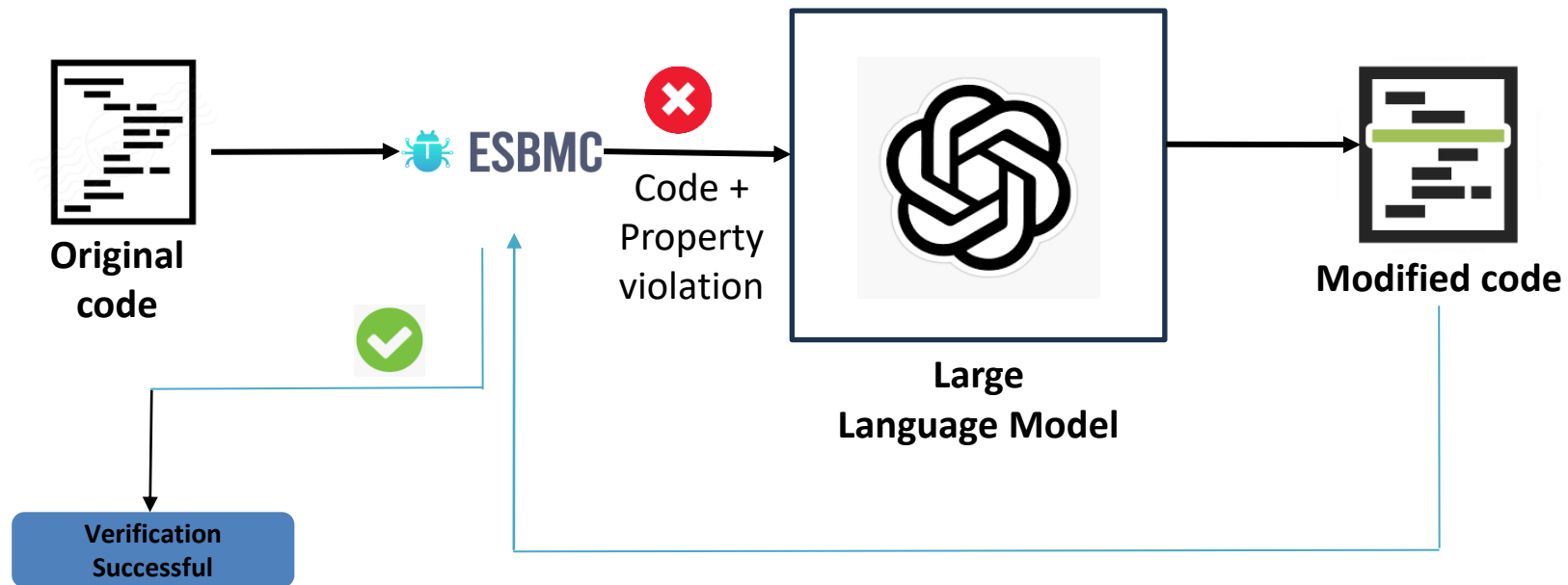


LLM + Formal Verification for Self-Healing Software



[6] Charalambous, Y., Tihanyi, N., Jain, R., Sun, Y., Ferrag, M. Cordeiro, L.: A New Era in Software Security: Towards Self-Healing Software via Large Language Models and Formal Verification. Under review at the ACM Transactions on Software Engineering and Methodology, 2023.

LLM + Formal Verification for Self-Healing Software



LLM to Find Software Vulnerabilities

C++ program example

```
int main() {  
    int x=77;  
    int y=x*x*x;  
    int z=y*y;  
    unsigned int r= z/1000;  
    printf("Result %d\n", r);  
    return 0;  
}
```



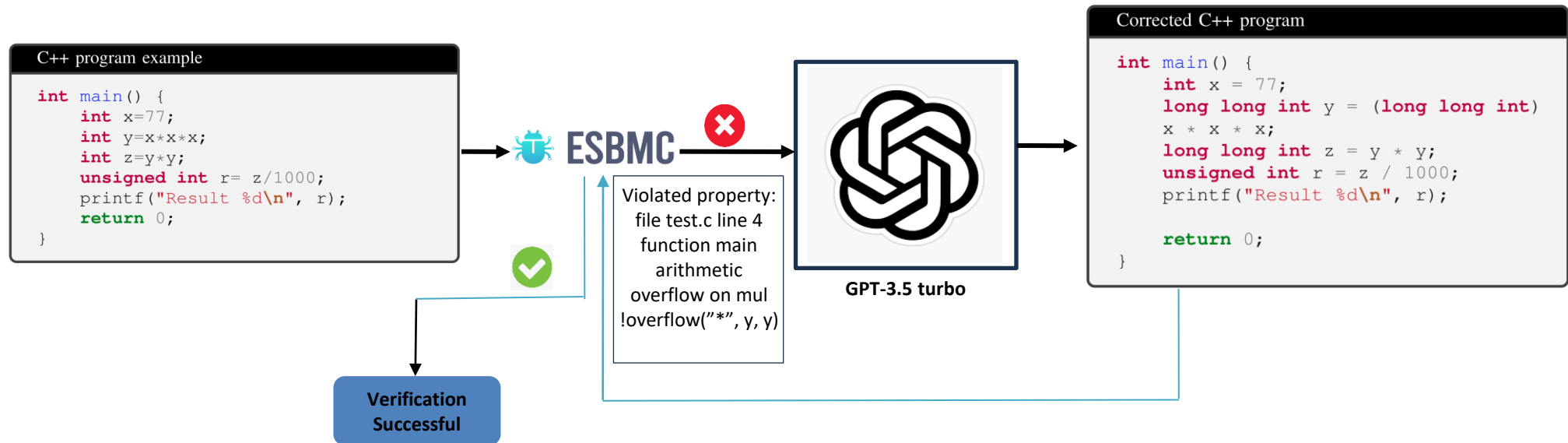
GPT-3.5 turbo

While we were in the process of preparing this presentation, if we asked GPT-3.5 *"Is there any problem with this code?"*, the response was an incorrect answer:

LLM answer:

"There is no problem with this code. The code correctly calculates the value of z as the square of y and then calculates the value of r as z divided by 1000. Finally, the code prints the result of r to the console. The calculations do not result in an integer overflow or any other issues, and the program should run correctly."

LLM + Formal Verification for Self-Healing Software



Experimental Evaluation

Set-up

Code Generation

- **Processor:** AMD Ryzen Threadripper PRO 3995WX
- **Cores:** 16
- **RAM:** 256 GB

Code Repair

- **Model:** MacBook Pro (2017)
- **RAM:** 16 GB RAM of LPDDR3 RAM (2133 MHz)
- **Processor:** 2.5 GHz Intel Core i7-7660U

Benchmarks

Generate 1000 programs with GPT-3.5 turbo with the following prompt

Code generation prompt

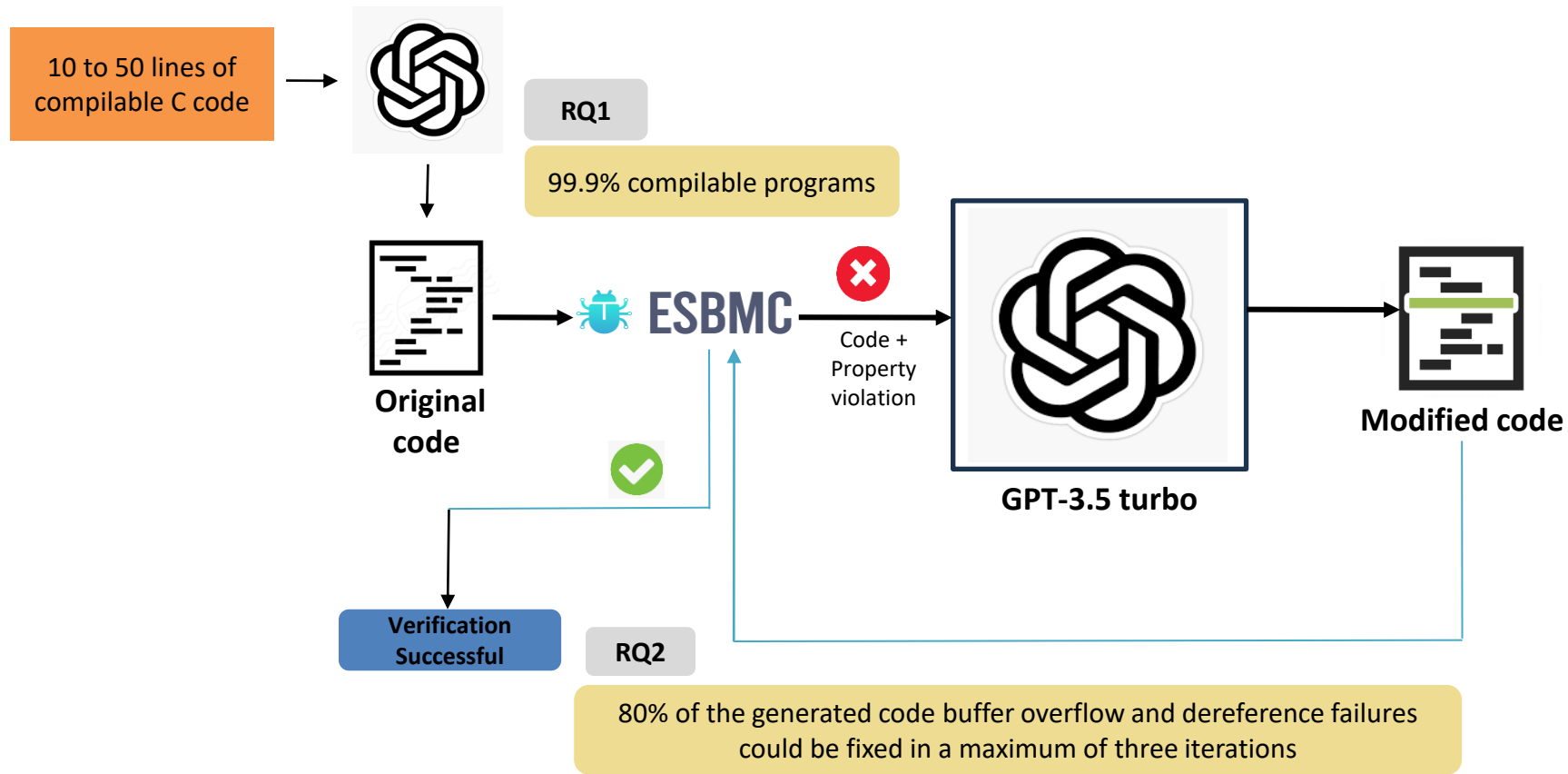
Generate a minimum of 10 and a maximum of 50 lines of C code. Use at least two functions. Use strings, arrays, bit manipulations, and string manipulations inside the code. Be creative! Always include every necessary header. Only give me the code without any explanation. No comment in the code.

Objectives

RQ1: (Code generation) Are the state-of-the-art GPT models capable of producing compilable, semantically correct programs?

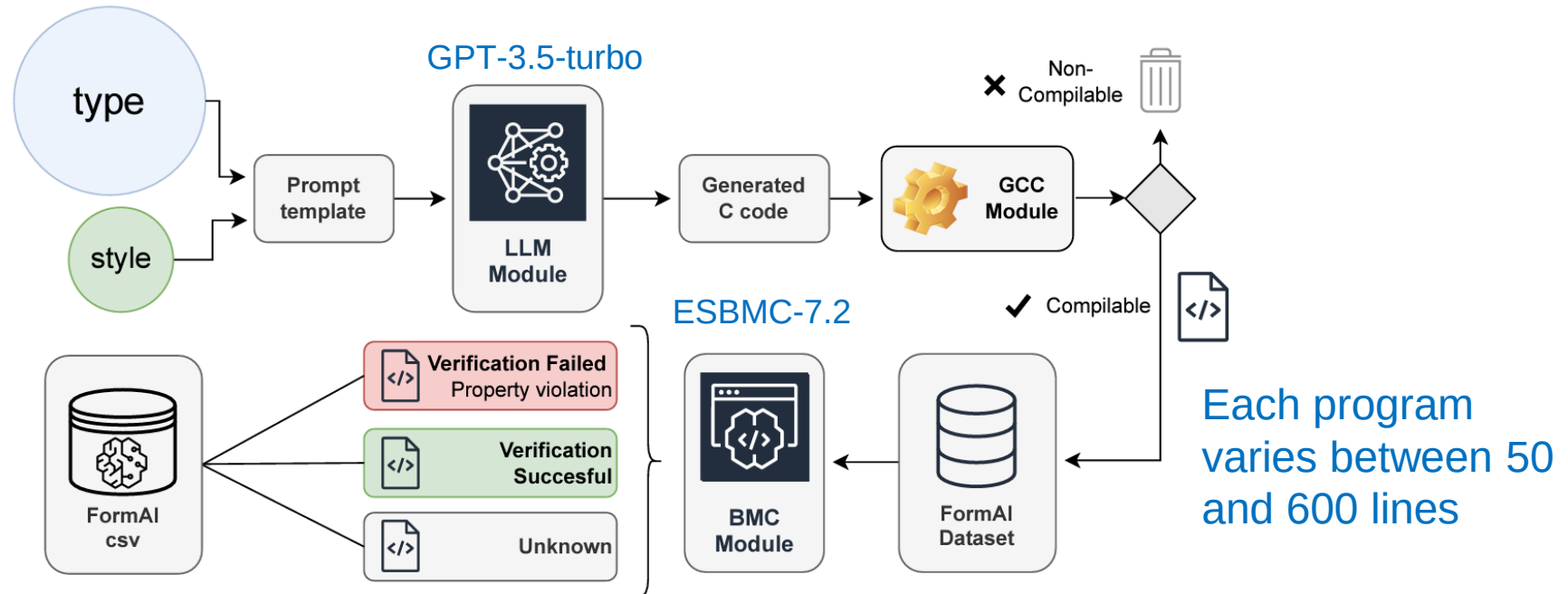
RQ2: (Code repair) Can external feedback improve the bug detection and patching ability of the GPT models?

Experimental Results



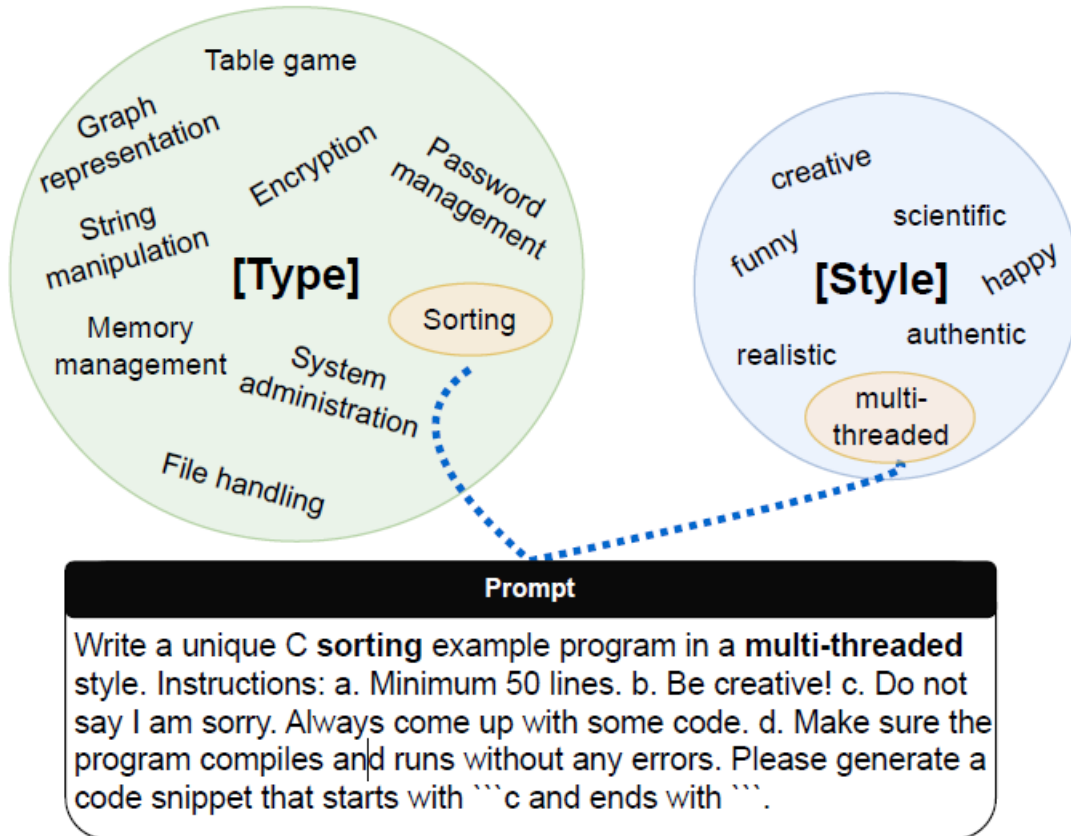
Generative AI through the Lens of Formal Verification

- The first AI-generated repository consisting of **112k independent and compilable C programs**



- Programming tasks from **network management and table games to string manipulation**

Ensure Diversity



- Proper prompt engineering is crucial for achieving a diverse dataset
- Each API call randomly chooses a type from 200 options in the Type category, including topics like Wi-Fi Signal Strength Analyzer, QR Code Reader, and others
- Similarly, a coding style is selected from 100 options in the Style category during each query

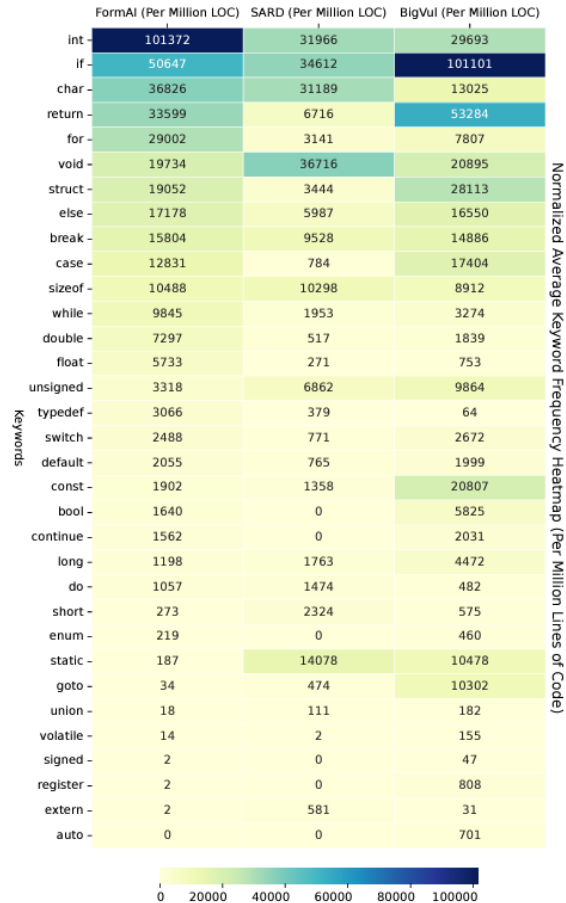
Comparison of Various Datasets Based on their Labeling Classifications

Dataset	Only C	Source	#Code Snips.	#Vuln. Snips.	Multi. Vulns/Snip.	Comp./Gran.	Vuln. Label.	Avg. LOC	Label. Method
Big-Vul	✗	Real-World	188,636	100%	✗	✗/Func.	CVE/CVW	30	PATCH
Draper	✗	Syn.+Real-World	1,274,366	5.62%	✓	✗/Func.	CWE	29	STAT
SARD	✗	Syn.+Real-World	100,883	100%	✗	✓/Prog.	CWE	114	BDV+STAT+MAN
Juliet	✗	Synthetic	106,075	100%	✗	✓/Prog.	CWE	125	BDV
Devign	✗	Real-World	27,544	46.05%	✗	✗/Func.	CVE	112	ML
REVEAL	✗	Real-World	22,734	9.85%	✗	✗/Func.	CVE	32	PATCH
DiverseVul	✗	Real-World	379,241	7.02%	✗	✗/Func.	CWE	44	PATCH
FormAI	✓	AI-gen.	112,000	51.24%	✓	✓/Prog.	CWE	79	ESBMC

Legend:

PATCH: GitHub Commits Patching a Vuln. **Man**: Manual Verification, **Stat**: Static Analyser, **ML**: Machine Learning Based, **BDV**: By design vulnerable

C Keyword Frequency and Associated CWEs



#Vulns	Vuln.	Associated CWE-numbers
88,049	<i>BOF</i>	CWE-20, CWE-120, CWE-121, CWE-125, CWE-129, CWE-131, CWE-628, CWE-676, CWE-680, CWE-754, CWE-787
31,829	<i>DFN</i>	CWE-391, CWE-476, CWE-690
24,702	<i>DFA</i>	CWE-119, CWE-125, CWE-129, CWE-131, CWE-755, CWE-787
23,312	<i>ARO</i>	CWE-190, CWE-191, CWE-754, CWE-680, CWE-681, CWE-682
11,088	<i>ABV</i>	CWE-119, CWE-125, CWE-129, CWE-131, CWE-193, CWE-787, CWE-788
9823	<i>DFI</i>	CWE-416, CWE-476, CWE-690, CWE-822, CWE-824, CWE-825
5810	<i>DFE</i>	CWE-401, CWE-404, CWE-459
1620	<i>OTV</i>	CWE-119, CWE-125, CWE-158, CWE-362, CWE-389, CWE-401, CWE-415, CWE-459, CWE-416, CWE-469, CWE-590, CWE-617, CWE-664, CWE-662, CWE-685, CWE-704, CWE-761, CWE-787, CWE-823, CWE-825, CWE-843
1567	<i>DBZ</i>	CWE-369

- $ARO \subseteq VF$: Arithmetic overflow
- $BOF \subseteq VF$: Buffer overflow on `scanf()`/`fscanf()`
- $ABV \subseteq VF$: Array bounds violated
- $DFN \subseteq VF$: Dereference failure : NULL pointer
- $DFE \subseteq VF$: Dereference failure : forgotten memory
- $DFI \subseteq VF$: Dereference failure : invalid pointer
- $DFA \subseteq VF$: Dereference failure : array bounds violated
- $DBZ \subseteq VF$: Division by zero
- $OTV \subseteq VF$: Other vulnerabilities

The CWE Top 13

#	ID	Name
1	CWE-787	Out-of-bounds Write
2	CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
3	CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
4	CWE-20	Improper Input Validation
5	CWE-125	Out-of-bounds Read
6	CWE-78	Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
7	CWE-416	Use After Free
8	CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
9	CWE-352	Cross-Site Request Forgery (CSRF)
10	CWE-434	Unrestricted Upload of File with Dangerous Type
11	CWE-476	NULL Pointer Dereference
12	CWE-502	Deserialization of Untrusted Data
13	CWE-190	Integer Overflow or Wraparound

Which Parameters Are Most Effective?

Table: Classification results for different parameters

(u,t)	VULN	k-ind	Running time (m:s)	$\mathcal{VS}$	$\mathcal{VF}$	$\mathcal{TO}$	$\mathcal{ER}$
(2,1000)	2438	$\times$	758:09	371	547	34	48
(3,1000)	2373	$\times$	1388:39	366	527	57	50
(2,100)	2339	$\times$	175:38	367	529	61	43
(2,100)	2258	$\checkmark$	400:54	340	603	20	37
(1,100)	2201	$\times$	56:29	416	531	17	36
(1,30)	2158	$\checkmark$	146:13	349	581	34	36
(3,100)	2120	$\times$	284:22	354	483	120	43
(1,30)	2116	$\times$	30:57	416	519	30	35
(1,10)	2069	$\checkmark$	61:58	360	553	52	35
(1,10)	2038	$\times$	19:32	413	503	51	33
(3,30)	1962	$\times$	125:19	342	444	172	42
(1,1)	1557	$\checkmark$	10:59	355	406	208	31
(1,1)	1535	$\times$	6:22	395	374	201	30

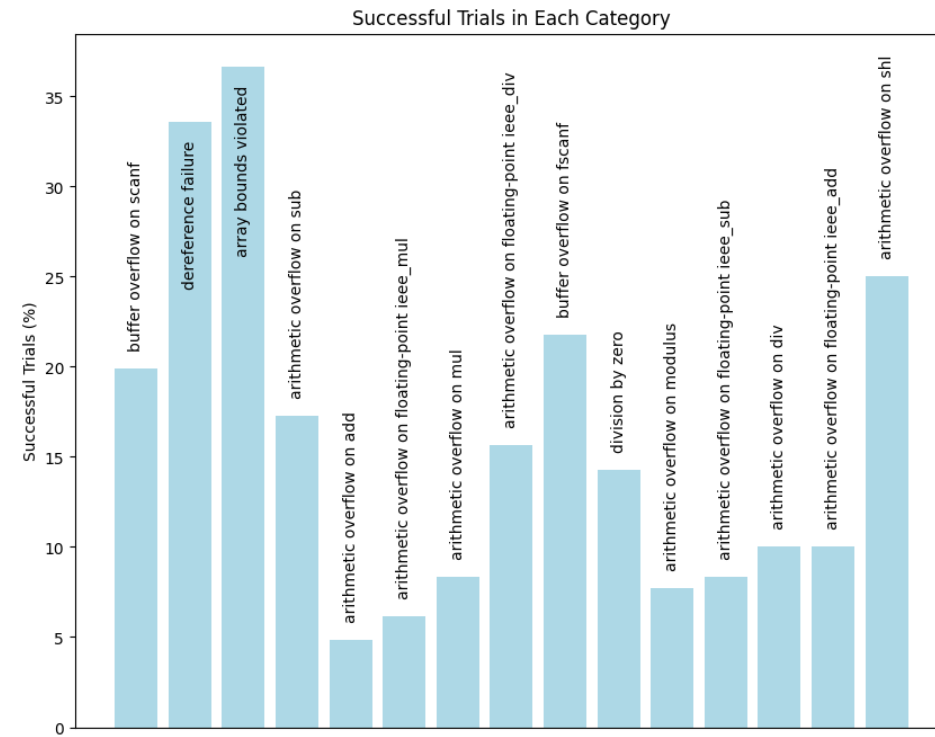
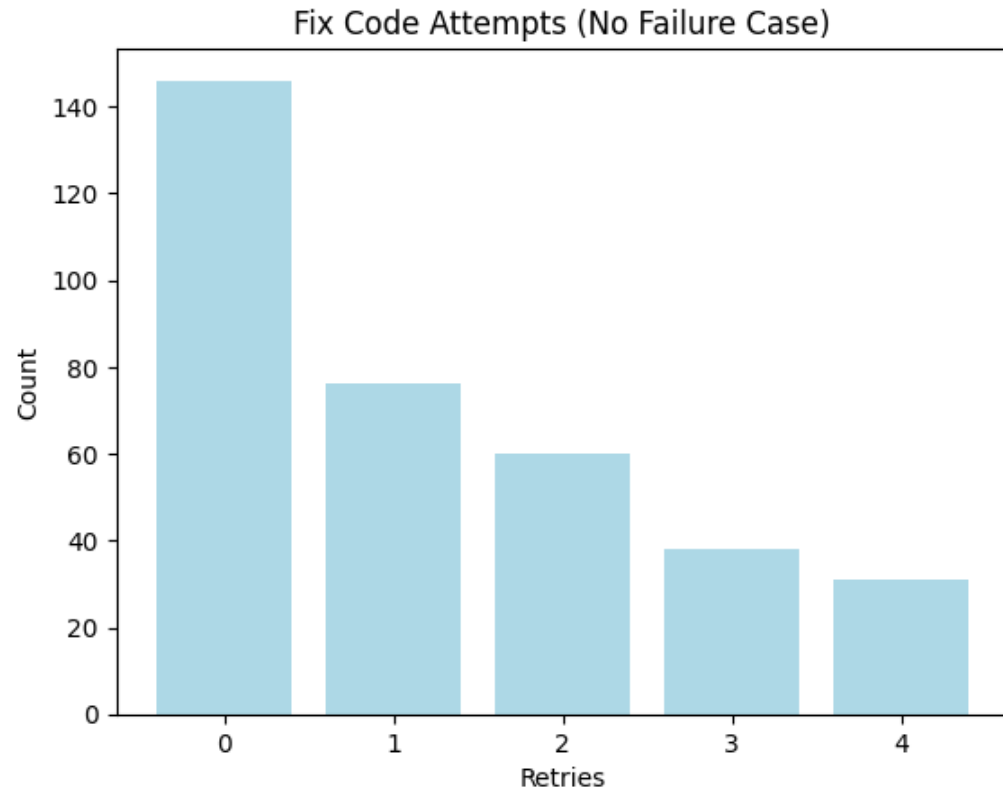
$\checkmark$: Enabled, $\times$: Disabled, (u, t) = unwind and timeout parameters

- We conducted experiments on 1,000 randomly selected samples
- The classification results showcase the effects of different unwind (u) and time (t) coupled with/without k-induction
- The detection results for parameter selection of $(u,t)=(1,10)$, $(1,30)$, or $(1,100)$ without k-induction show that increasing the time threshold yields diminishing returns for the same unwind parameter

```
esbmc file.c --overflow --unwind 1 --memory-leak-check
--timeout 30 --multi-property --no-unwinding-assertions
```

Code Repair Performance

FormAI dataset	Accuracy
1000 samples randomly selected from 112k C programs	35.5%



FormAI Dataset - Availability

FORMAI DATASET: A LARGE COLLECTION OF AI-GENERATED C PROGRAMS AND THEIR VULNERABILITY CLASSIFICATIONS



☆☆☆☆☆ 0 ratings - Please [login](#) to submit your rating.

Citation Author(s): Norbert Tihanyi  (Technology Innovation Institute)
Tamas Bisztray  (University of Oslo)
Ridhi Jain  (Technology Innovation Institute)
Mohamed Amine Ferrag  (Technology Innovation Institute)
Lucas C. Cordeiro  (University of Manchester)
Vasileios Mavroeidis  (University of Oslo)

Submitted by: Norbert Tihanyi

Last updated: Tue, 09/26/2023 - 05:10

DOI: [10.21227/vp9n-wv96](https://doi.org/10.21227/vp9n-wv96)

Data Format: *.csv (zip);

License: Creative Commons Attribution 

[↓ ACCESS DATASET](#) [📄 CITE](#) [🔗 SHARE/EMBED](#)

 827 Views

Categories: [Artificial Intelligence](#)
[Security](#)

Keywords: [artificial intelligence](#), [Software Vulnerability Dataset](#)

WARNING: BE CAREFUL WHEN RUNNING THE COMPILED PROGRAMS, SOME CAN CONNECT TO THE WEB, SCAN YOUR LOCAL NETWORK, OR DELETE A RANDOM FILE FROM YOUR FILE SYSTEM. ALWAYS CHECK THE SOURCE CODE AND THE COMMENTS IN THE FILE BEFORE RUNNING IT!!!

<https://github.com/FormAI-Dataset>

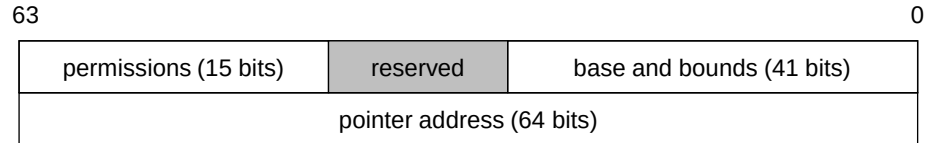
DATASET FILES

- FormAI dataset: Vulnerability Classification (No C source code included) FormAI_dataset_human_readable-V1.csv (15.95 MB)
- FormAI dataset: 112000 compilable AI-generated C code FormAI_dataset_C_samples-V1.zip (97.61 MB)
- FormAI dataset: Vulnerability Classification (C source code included in CSV) FormAI_dataset_classification-V1.zip (60.66 MB)

Agenda

- Software Verification and Testing with the ESBMC Framework
- Towards Self-Healing Software via Large Language Models and Formal Verification
- Towards verification of programs for CHERI Platforms with ESBMC

Capability Hardware Enhanced RISC Instructions (CHERI)



CHERI 128-bit capability

CHERI Clang/LLVM and LLD¹ - compiler and linker for CHERI ISAs

¹<https://www.cl.cam.ac.uk/research/security/ctsr/cheri/cheri-llvm.html>

CheriBSD² - adaptation of FreeBSD to support CHERI ISAs

²<https://www.cl.cam.ac.uk/research/security/ctsr/cheri/cheribsd.html>

ARM Morello³ - SoC development board with a CHERI-extended ARMv8-A processor

³<https://www.arm.com/architecture/cpu/morello>

Mnemonic	Description
CGetBase	Move base to a GPR
CGetLen	Move length to a GPR
CGetTag	Move tag bit to a GPR
CGetPerm	Move permissions to a GPR
CGetPCC	Move the PCC and PC to GPRs
CIncBase	Increase base and decrease length
CSetLen	Set (reduce) length
CClearTag	Invalidate a capability register
CAndPerm	Restrict permissions
CToPtr	Generate C0-based integer pointer from a capability
CFromPtr	CIncBase with support for NULL casts
CBTU	Branch if capability tag is unset
CBTS	Branch if capability tag is set
CLC	Load capability register



CHERI-C program

```
#include <stdlib.h>
#include <string.h>
#include <cheri/cheric.h>

void main() {
    int n = nondet_uint() % 1024;
    char a[n+1], *__capability b = cheri_ptr(a, n+1);
    b[n] = 17;
    char *__capability c = cheri_setbounds(b-1, n);
    /* ... */
    memset_c(c, 42, n);
}
```

CHERI-C API

/* models arbitrary user input */

/* succeeds */

/* fails: not the same object */

/* more CHERI-C API checks */

/* setting memory through a capability */

New capability types

Pure-capability CHERI-C model

```
#include <stdlib.h>
#include <string.h>
#include <cheri/cheric.h>
```

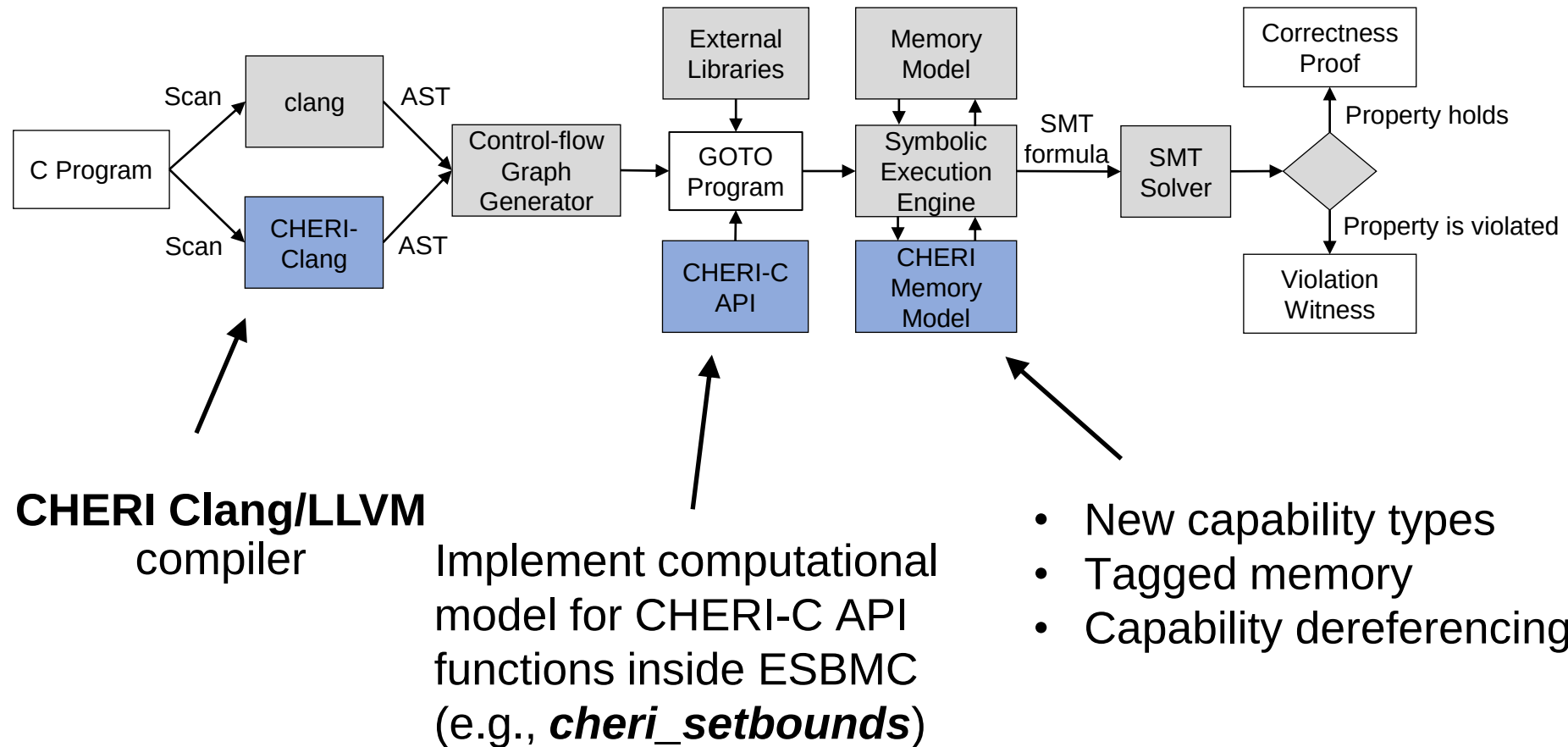
```
void main() {
    int n = nondet_uint() % 1024;
    char a[n+1], *__capability b = cheri_ptr(a, n+1);
    b[n] = 17;
    char *__capability c = cheri_setbounds(b-1, n);
    /* ... */
    memset_c(c, 42, n);
}
```

```
#include <string.h>
#include <stdio.h>
```

```
void main(void) {
    int n = nondet_uint() % 1024;
    char a[n+1], *b = a;
    b[n] = 17;
    char *c = b-1;
    memset(c, 42, n);
}
```

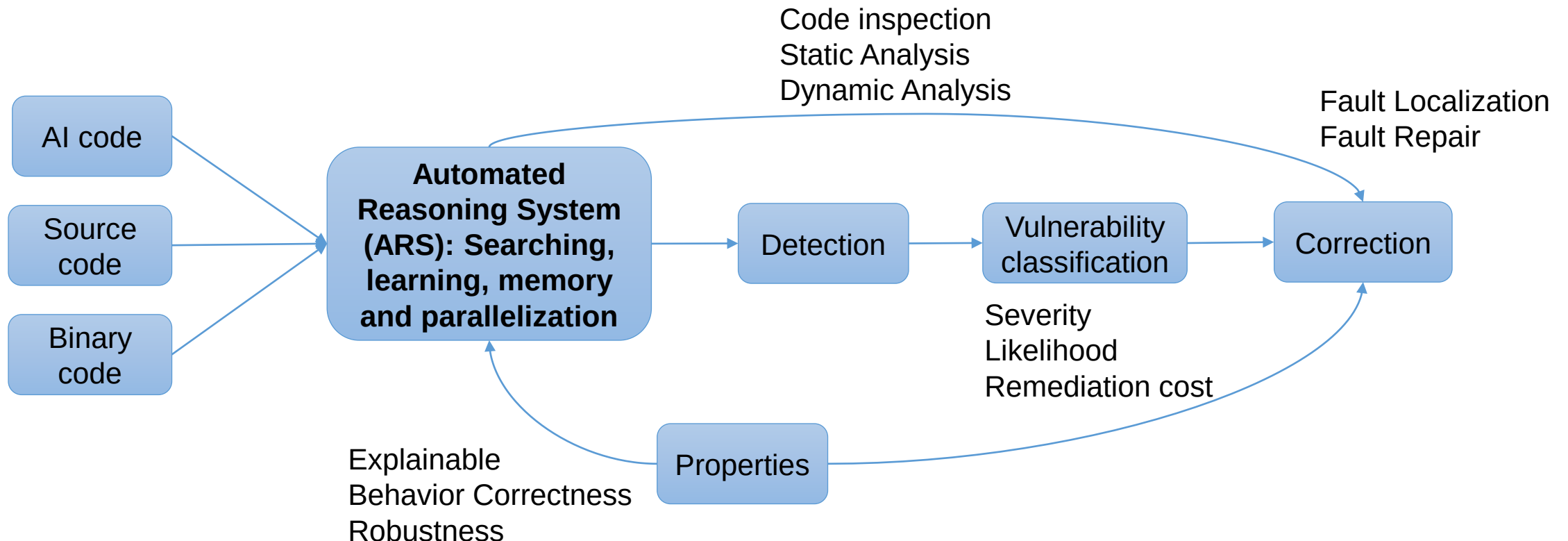
All pointers are automatically replaced with capabilities by the **CHERI Clang/LLVM** compiler

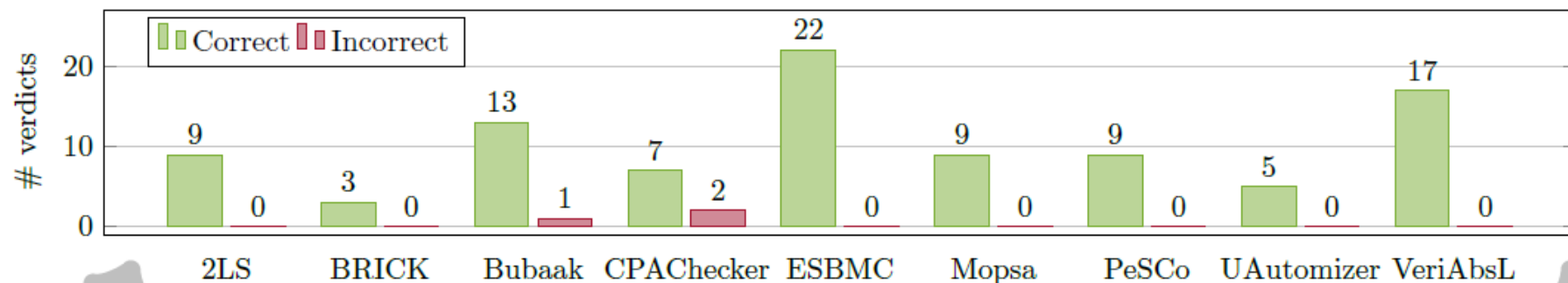
ESBMC-CHERI



Vision: Automated Reasoning System for Secure SW and AI

Develop an automated reasoning system for safeguarding software and AI systems against security vulnerabilities in an increasingly digital and interconnected world





Preliminary results
from SV-COMP'24
(may still change)

Existing software verifiers struggle on neural code!

Missing verdicts
due to timeouts
on many instances

Neural code challenges existing software verifiers:
float operations, calls to `math.h`, nested loops, multi-dimensional arrays.

We release *NeuroCodeBench*, a benchmark of neural code verification:
6 categories, 14 functions from `math.h`, 32 neural networks, 607 properties;
safe/unsafe verdicts are either known a priori or independently verified.

Benchmark Category	Safe	Unsafe
<code>math_functions</code>	33	11
<code>activation_functions</code>	40	16
<code>hopfield_nets</code>	47	33
<code>poly_approx</code>	48	48
<code>reach_prob_density</code>	22	13
<code>reinforcement_learning</code>	103	193
Total	293	314

(Real) Impact: Students and Contributors

- 5 PhD theses
- 30+ MSc dissertations
- 30+ final-year projects
- GitHub:
 - 35 contributors
 - 22,160 commits
 - 212 stars
 - 84 forks



<https://github.com/esbmc/esbmc>

Impact: Awards and Industrial Deployment

- Distinguished Paper Award at ICSE'11
- Best Paper Award at SBESC'15
- Most Influential Paper Award at ASE'23
- Best Tool Paper Award at SBSeg'23
- **29 awards** from the international competitions on software verification (SV-COMP) and testing (Test-Comp) 2012-2023 at **TACAS/FASE**
 - Bug Finding and Code Coverage 
- **Intel** deploys **ESBMC** in production as one of its verification engines for **verifying firmware in C**
- **Nokia and ARM** have found **security vulnerabilities** in **C/C++ software**
- **Funded by government** (EPSRC, British Council, Royal Society, CAPES, CNPq, FAPEAM) and **industry** (Intel, Motorola, Samsung, Nokia, ARM)

Acknowledgements

